gemstone-rs Companion Manual

Setup, user manual, examples, cookbook, codegen, explorer, and workbench in one PDF

Generated from the Markdown sources in gemstone-rs/docs.

Contents

Post-Release Verification

Optional Live Smoke

gemstone-rs Documentation

This directory contains the human-facing guides for `gemstone-rs`.

Start Here

Guide	Use it when
Setup Guide	You need Rust, GemStone environment variables, install commands, and first login checks.
Examples Guide	You want to know which example to run for each feature.
User Manual	You want the main Rust API surface in one place.
Cookbook	You want task-focused recipes for sessions, transactions, browser calls, codegen, explorer, and VS Code.
gemstone-py vs gemstone-rs	You want install paths, use cases, maturity, and feature differences between the Python and Rust projects.
BridgeRoot and Object Mapping	You want MagLev-style bridge-root storage and explicit Rust-to-GemStone value mapping.
Codegen Guide	You want generated Rust wrappers for GemStone classes and methods.
Codegen Profile Schema	You want project profile JSON validation for explorer and VS Code workflows.
Explorer Guide	You want the local HTTP explorer API and safety defaults.
VS Code Workbench	You want sidebar browsing, codegen preview/diff/generate, and explorer launch commands.
Screenshot Workflow	You want to refresh Explorer and Workbench images before docs or Marketplace releases.
Performance and Safety	You want benchmark guidance, GCI loading notes, and threading rules.
Shared Core Integration	You want the plan for `gemstone-py-native` to wrap the Rust core.
Medium Article	You want an article-style explanation suitable for publishing or sharing.

Guide	Use it when
Funny Introduction	You want a lighter multi-part tour of the same concepts.
Release Checklist	You want the crate, VSIX, GitHub Release, and post-release verification flow.

PDFs

Generated PDFs live in pdf/. Rebuild them after Markdown changes:

```
python3 docs/build_pdf_docs.py
```

Refresh screenshots before a visual release:

```
make screenshots
```

Shortest Path

```
cargo install gemstone-rs-cli
gemstone-rs --help
```

For library use:

```
cargo add gemstone-rs
```

For local examples from a checkout:

```
cd /path/to/gemstone-rs
cargo run -p gemstone-rs --example quickstart
```

For the local explorer:

```
cargo install gemstone-rs-explorer
gemstone-rs-explorer --port 8787
```

For VS Code:

```
https://marketplace.visualstudio.com/items?itemName=unicompute.gemstone-rs-workbench
```

Setup Guide

This guide gets you from a Rust project or source checkout to a live GemStone login.

What You Need

At minimum:

- Rust stable toolchain
- access to a GemStone/S 64 stone
- GemStone client library access through `libgcirpc`
- GemStone username and password

For the full local development experience:

- a `gemstone-rs` checkout
- `cargo`, `rustfmt`, and `clippy`
- Node.js 22 when working on the VS Code extension
- a Marketplace PAT only when publishing the VSIX
- a crates.io API token only when publishing crates

Which Install Path Should I Use?

Use case	Command
Rust application dependency	`cargo add gemstone-rs`
CLI tools	`cargo install gemstone-rs-cli`
Local HTTP explorer	`cargo install gemstone-rs-explorer`
Source checkout examples	`cargo run -p gemstone-rs --example quickstart`
VS Code users	Install `unicompute.gemstone-rs-workbench` from Marketplace

Environment Variables

Most live commands use `Config::from_env()`:

```
gemstone-rs env sample
export GS_LIB=/opt/gemstone/product/lib
export GS_STONE=gs64stone
export GS_STONE_NAME=gs64stone
export GS_USERNAME=DataCurator
export GS_PASSWORD=swordfish
```

`GS_STONE` is canonical. `GS_STONE_NAME` is accepted as an alias when `GS_STONE` is absent. Setting both to the same value keeps CLI, examples, explorer, and VS Code settings aligned.

`gemstone-rs env sample` prints a safe shell export template. It carries over current non-secret values such as `GS_LIB` or `GS_STONE`, comments optional values when they are absent, and prints placeholders for `GS_PASSWORD` and `GS_HOST_PASSWORD` instead of copying secrets.

Use `gemstone-rs env write` to save the same template:

```
gemstone-rs env write
gemstone-rs env write .env.gemstone-rs --force
```

It refuses to overwrite an existing file unless `--force` is passed. Use the file directly without sourcing it:

```
gemstone-rs doctor --env-file .env.gemstone-rs
gemstone-rs doctor --env-file .env.gemstone-rs --live
gemstone-rs eval --env-file .env.gemstone-rs "3 + 4"
gemstone-rs --env-file .env.gemstone-rs browse dictionaries
gemstone-rs --env-file .env.gemstone-rs codegen check gemstone-rs.codegen
gemstone-rs-explorer --env-file .env.gemstone-rs --port 8787
```

Optional variables:

```
export GS_HOST=localhost
export GS_NETLDI=netldi
export GS_GEM_SERVICE=gemnetobject
export GS_HOST_USERNAME=
export GS_HOST_PASSWORD=
export GS_LIB_PATH=/full/path/to/libgcirpc.dylib
```

Variable	Required	Purpose
`GS_LIB`	Usually	GemStone `lib/` directory for runtime library discovery.

Variable	Required	Purpose
`GS_LIB_PATH`	No	Full path to a specific `libgcirpc` file.
`GS_STONE`	Yes	Stone name.
`GS_STONE_NAME`	No	Stone-name alias used when `GS_STONE` is absent.
`GS_USERNAME`	Yes	GemStone login username.
`GS_PASSWORD`	Yes	GemStone login password.
`GS_HOST`	No	Remote host, defaults to local behavior.
`GS_NETLDI`	No	NetLDI service name.
`GS_GEM_SERVICE`	No	Gem service name.

First Login

With the CLI:

```
cargo install gemstone-rs-cli
gemstone-rs env sample
gemstone-rs doctor
gemstone-rs doctor --live
gemstone-rs doctor --strict
gemstone-rs doctor --env-file .env.gemstone-rs --live
gemstone-rs --env-file .env.gemstone-rs codegen explain --json gemstone-rs.codegen
gemstone-rs doctor --json
gemstone-rs eval "3 + 4"
```

Expected output:

```
7
```

`gemstone-rs doctor` prints the relevant `GS_*` values with secrets masked, loads the configured `libgcirpc`, and reports setup problems without exposing passwords. The GCI section reports the source that selected the library: explicit config, `GS_LIB_PATH`, `GS_LIB`, or `GEMSTONE/lib`, plus the exact path or directory searched. Add `--live` when the stone should be reachable; it logs in and asserts that `3 + 4` returns `7`. Add `--json` when a script, CI job, or editor integration needs a parseable report. Failed checks include hints for missing credentials, `libgcirpc` path/loading problems, and live stone connectivity. `--strict` is useful for CI because it also fails when

`GS_STONE` / `GS_STONE_NAME` or a deterministic GCI library source (`GS_LIB_PATH`, `GS_LIB`, or `GEMSTONE`) is missing. The GCI section reports whether the selected `libgcirpc` path exists, is a file, is readable, and whether the path appears to reference arm64 or x86_64.

From a source checkout:

```
cd /path/to/gemstone-rs
cargo run -p gemstone-rs --example hello_gemstone
cargo run -p gemstone-rs --example quickstart
```

From a Rust application:

```
use gemstone_rs::{Config, Session, Value};

fn main() -> gemstone_rs::Result<()> {
    let mut session = Session::login(Config::from_env()?)?;
    assert_eq!(session.eval("3 + 4")?, Value::SmallInt(7));
    Ok(())
}
```

Development Checkout

```
git clone git@github.com:unicompute/gemstone-rs.git
cd gemstone-rs
make verify
```

`make verify` runs formatting, `cargo check`, clippy, tests, codegen freshness, the VS Code extension syntax/smoke checks, and PDF generation.

Package the VSIX locally:

```
make vscode-package
```

Verify already-published artifacts:

```
scripts/publish_verify.sh 0.2.2
```

User Manual

`gemstone-rs` is the safe Rust client API for GemStone/S over GCI. Rust code imports it as `gemstone_rs`.

Configuration

Use `Config::from_env()` for normal applications:

```
let config = gemstone_rs::Config::from_env()?;
```

Use the builder when you want explicit configuration:

```
use gemstone_rs::Config;

let config = Config::builder()
    .stone("gs64stone")
    .host("localhost")
    .netldi("netldi")
    .username("DataCurator")
    .password("swordfish")
    .build()?;
```

Sessions

`Session` logs out automatically on drop. Calling `logout()` explicitly is still useful in examples and command-line tools.

```
use gemstone_rs::{Config, Session, Value};

let mut session = Session::login(Config::from_env()?)?;
let value = session.eval("3 + 4")?;
assert_eq!(value, Value::SmallInt(7));
session.logout()?;
```

`Session` is deliberately not `Send` or `Sync`. Keep a session on the thread that logged it in.

Eval and Perform

`eval` returns a marshalled `Value`:

```
let value = session.eval("3 + 4")?;
```

`execute` returns a raw `Oop`:

```
let object_class = session.execute("Object")?;
```

`perform` returns a marshalled `Value`:

```
let seven = session.smallint_oop(7);
let printed = session.perform(seven, "printString", &[])?;
```

`perform_oop` returns a raw `Oop`:

```
let printed_oop = session.perform_oop(seven, "printString", &[])?;
let printed = session.fetch_string(printed_oop)?;
```

OOP and Value Conversion

`Value` covers nil, booleans, small integers, characters, fetched strings, and raw OOPs:

```
use gemstone_rs::Value;

let oop = session.value_to_oop(&Value::SmallInt(7))?;
let value = session.eval("true")?;
println!("{value:?}");
```

Helpers:

```
let nil = session.nil_oop();
let yes = session.bool_oop(true);
let seven = session.smallint_oop(7);
let text = session.new_string("hello")?;
let symbol = session.new_symbol("ExampleSymbol")?;
```

Globals

`global_get` and `global_put` use `UserGlobals`:

```
let text = session.new_string("hello from Rust")?;
session.global_put("GemStoneRsText", text)?;
```

```
let stored = session.global_get("GemStoneRsText")?;
println!("{}", session.fetch_string(stored)?);
```

Transactions

Use `transaction` when you want commit-on-success and abort-on-error:

```
session.transaction(|session| {
    let value = session.new_string("committed")?;
    session.global_put("GemStoneRsCommitted", value)
})?;
```

Manual transaction primitives are also available:

```
let needs_commit = session.needs_commit()?;
let in_transaction = session.in_transaction()?;
session.commit()?;
session.abort()?;
```

Export-Set Lifetime

Use `retain_oop` when a raw OOP must be held across calls and protected from GemStone-side collection:

```
let text = session.new_string("retained")?;
let handle = session.retain_oop(text)?;
println!("{}", handle.oop().raw());
handle.release()?;
```

The handle releases on drop if you do not call `release()`.

Browser API

```
use gemstone_rs::browser::{Browser, ALL_PROTOCOLS};

let mut browser = Browser::new(&mut session);
let dictionaries = browser.dictionaries()?;
let classes = browser.classes("UserGlobals")?;
let protocols = browser.protocols("Object", false, "")?;
let methods = browser.methods("Object", ALL_PROTOCOLS, false, "")?;
let source = browser.source("Object", "printString", false, "")?;
```

Pass an empty dictionary to resolve through the active user's symbol list.

Error Handling

Most APIs return `gemstone_rs::Result<T>`. Errors distinguish missing environment, missing config, GCI loading/calls, GemStone errors, illegal OOPs, unexpected typed codegen returns, and conversion issues.

```
match Config::from_env() {
    Ok(config) => println!("stone {}", config.stone),
    Err(err) => eprintln!("configuration error: {err}"),
}
```

The CLI exposes the same checks through `doctor`:

```
gemstone-rs env sample
gemstone-rs env write
gemstone-rs doctor
gemstone-rs doctor --live
gemstone-rs doctor --strict
gemstone-rs doctor --env-file .env.gemstone-rs --live
gemstone-rs doctor --json
gemstone-rs eval --env-file .env.gemstone-rs "3 + 4"
gemstone-rs --env-file .env.gemstone-rs browse dictionaries
gemstone-rs --env-file .env.gemstone-rs codegen check gemstone-rs.codegen
```

`env sample` prints a safe shell export template with password placeholders, and `env write` saves it to `.env.gemstone-rs` without overwriting existing files unless `--force` is used. `--env-file` is a global CLI option, so `doctor`, `eval`, `browse`, `inspect`, `bridge`, and `codegen` commands can load that file for one command. The non-live doctor form validates environment and GCI library loading. The live form also logs in and checks that `3 + 4` returns `7`. Human and JSON reports show which source selected `libgcirpc`: explicit config, `GS_LIB_PATH`, `GS_LIB`, or `GEMSTONE/lib`, plus the exact path or directory searched. `doctor --strict` fails when the stone or GCI source is only coming from defaults, which is better for CI. The JSON form is intended for automation and editor integrations, and includes the same remediation hints as the human report.

Examples Guide

The `gemstone-rs` examples are split between runnable Cargo examples and the checked-in codegen sample under the repository-level `examples/` directory.

Common Setup

```
gemstone-rs env sample
gemstone-rs env write
gemstone-rs doctor --env-file .env.gemstone-rs
gemstone-rs --env-file .env.gemstone-rs browse dictionaries
gemstone-rs --env-file .env.gemstone-rs codegen check examples/codegen/gemstone-
rs.codegen
export GS_LIB=/opt/gemstone/product/lib
export GS_STONE=gs64stone
export GS_STONE_NAME=gs64stone
export GS_USERNAME=DataCurator
export GS_PASSWORD=swordfish
```

Run Cargo examples from the repository root:

```
cd /path/to/gemstone-rs
cargo run -p gemstone-rs --example quickstart
```

Example Map

Feature	Command or path	What it demonstrates
First login	`cargo run -p gemstone-rs --example hello_gemstone`	Reads env config, logs in, prints a session id, and evaluates `3 + 4`.
Quickstart	`cargo run -p gemstone-rs --example quickstart`	Eval, `global_put`, `global_get`, string fetch, cleanup.
Eval only	`cargo run -p gemstone-rs --example eval`	Minimal `Session::eval` shape.
Browser API	`cargo run -p gemstone-rs --example browser`	Dictionaries, protocols, methods, and source.
Live smoke cookbook	`cargo run -p gemstone-rs --example live_smoke_cookbook`	Login, eval, global round-trip, perform, and transaction checks in one run.
Transactions	`cargo run -p gemstone-rs --example transactions`	Commit-on-success and abort-on-error transaction wrapper.
OOP/value conversion	`cargo run -p gemstone-rs --example oop_values`	`Value`, `Oop`, strings, symbols, and export-set retention.

Feature	Command or path	What it demonstrates
BridgeRoot mapping	`cargo run -p gemstone-rs --example bridge_root_mapping`	MagLev-style bridge-root storage with explicit `BridgeValue` mapping.
Derive mapping	`cargo run -p gemstone-rs --example derive_mapping`	`#[derive(BridgeMapped)]`, symbol keys, nested structs, vectors, maps, and BridgeRoot transactions.
Offline codegen	`cargo run -p gemstone-rs --example codegen_preview`	Generates wrappers from the sample config without a live stone.
Codegen workflow	`cargo run -p gemstone-rs --example codegen_workflow`	Writes config, previews, diffs, checks, generates, and verifies a clean diff.
Codegen discovery	`cargo run -p gemstone-rs --example codegen_discover`	Connects to a live stone and discovers a starter config for `Object`.
Mapping discovery	`cargo run -p gemstone-rs --example codegen_discover_mapping`	Connects to a live stone and proposes a `BridgeMapped` config.
Generated wrapper app	`cargo run -p gemstone-rs --example generated_wrapper_app`	Uses checked-in generated wrappers to call `Object>>printString`.
Generated mapping app	`cargo run -p gemstone-rs --example generated_mapping_app`	Uses codegen-created `BridgeMapped` structs with `BridgeRoot`.
Generated wrapper compile check	`cargo test --manifest-path examples/codegen-wrapper-check/Cargo.toml`	Imports the checked-in generated wrappers as a separate crate.
Codegen files	`examples/codegen/`	Config, generated wrappers, check/diff/generate workflow.
Explorer tooling	`examples/tooling/explorer.md`	Local explorer startup and endpoint checks.
VS Code tooling	`examples/tooling/vscode-workbench.md`	Sidebar browsing, codegen actions, and explorer launch.
CLI browser walkthrough	`examples/tooling/cli-browser-walkthrough.md`	Terminal-only browse workflow.
Axum service sketch	`examples/axum-service/README.md`	Recommended web-service shape without adding workspace dependencies.

Suggested Learning Order

1. `hello_gemstone`
2. `quickstart`
3. `browser`
4. `live_smoke_cookbook`
5. `oop_values`
6. `transactions`
7. `bridge_root_mapping`
8. `derive_mapping`
9. `codegen_preview`
10. `codegen_workflow`
11. `generated_wrapper_app`
12. `generated_mapping_app`
13. `codegen_discover`
14. `codegen_discover_mapping`
15. `examples/codegen/`
16. `examples/tooling/cli-browser-walkthrough.md`
17. `examples/tooling/explorer.md`
18. `examples/tooling/vscode-workbench.md`
19. `examples/axum-service/README.md`

Expected Output

Live examples need GemStone credentials and a reachable stone. If the environment is configured correctly, these are the important lines to look for:

```
$ cargo run -p gemstone-rs --example hello_gemstone
session id: <number>
3 + 4 => SmallInt(7)

$ cargo run -p gemstone-rs --example quickstart
GemStone eval ok: SmallInt(7)
GemStoneRsQuickstart: hello from gemstone-rs quickstart

$ cargo run -p gemstone-rs --example generated_wrapper_app
generated wrapper printString: 7

$ cargo run -p gemstone-rs --example live_smoke_cookbook
login ok: session <number>
eval ok: SmallInt(7)
global round-trip ok
perform ok: 7
transaction commit/abort ok

$ cargo run -p gemstone-rs --example bridge_root_mapping
bridge root: GemStoneRsBridgeRoot
```

```
MyTestDict OOP: <number>
loaded payload: BookingDraft { name: "Tariq", amount: 100, currency: "GBP", labels:
{"source": "manual"} }
loaded status: ready
loaded amount: 100
loaded approved: true
loaded tags: ["priority", "demo"]
loaded note: Some("front desk")
loaded labels: {"source": "manual"}
loaded symbol labels: {"source": "manual"}

$ cargo run -p gemstone-rs --example derive_mapping
derived mapped payload: BookingDraft { amount: 100, customer: CustomerDraft { name:
"Tariq" }, tags: ["priority", "demo"], labels: {"source": "derive"}, note: None }

$ cargo run -p gemstone-rs --example generated_mapping_app
generated mapped payload: BookingDraft { name: "Tariq", amount: 100, currency: "GBP",
tags: ["priority", "demo"], labels: {"source": "generated"}, note: Some("window
seat") }
```

Offline examples should run without GemStone:

```
$ cargo run -p gemstone-rs --example codegen_workflow
before generate: exists=false up_to_date=false diff_bytes=<number>
after generate: exists=true up_to_date=true
diff after generate: clean
```

CLI Equivalents

The CLI gives you the same live checks without compiling examples:

```
cargo install gemstone-rs-cli
gemstone-rs doctor
gemstone-rs doctor --live
gemstone-rs doctor --strict
gemstone-rs doctor --json
gemstone-rs eval --env-file .env.gemstone-rs "3 + 4"
gemstone-rs browse dictionaries
gemstone-rs browse classes UserGlobals
gemstone-rs browse protocols Object
gemstone-rs browse methods Object "-- all --"
gemstone-rs browse source Object printString
gemstone-rs inspect oop 20
gemstone-rs bridge root
gemstone-rs bridge keys
gemstone-rs bridge put-string WorkbenchDraft "hello from Rust"
gemstone-rs bridge put-symbol WorkbenchState ready
gemstone-rs bridge put-smallint WorkbenchCount 7
gemstone-rs bridge put-bool WorkbenchReady true
gemstone-rs bridge remove WorkbenchDraft
```

```
gemstone-rs bridge sample-config BookingDraft
gemstone-rs codegen explain examples/codegen/gemstone-rs.codegen
gemstone-rs codegen explain --json examples/codegen/gemstone-rs.codegen
gemstone-rs codegen explain-profile --json default examples/codegen/gemstone-
rs.codegen-profiles.json
cargo test --manifest-path examples/codegen-wrapper-check/Cargo.toml
```

Codegen Workflow

```
cargo run -p gemstone-rs-cli -- codegen init examples/codegen/demo.codegen
cargo run -p gemstone-rs-cli -- codegen preview examples/codegen/gemstone-rs.codegen
cargo run -p gemstone-rs-cli -- codegen diff examples/codegen/gemstone-rs.codegen
cargo run -p gemstone-rs-cli -- codegen check examples/codegen/gemstone-rs.codegen
cargo run -p gemstone-rs-cli -- codegen generate examples/codegen/gemstone-rs.codegen
cargo run -p gemstone-rs-cli -- codegen discover-mapping examples/codegen/
mapping.codegen BookingDraft Object
```

Use the checked-in explorer profile sample when you want a repeatable browser workflow for codegen:

```
cat examples/codegen/gemstone-rs.codegen-profiles.json
gemstone-rs-explorer --port 8787 --codegen-root .
```

In the explorer, click `Load Project Profiles` with `examples/codegen/gemstone-rs.codegen-profiles.json` as the project profile file, then select `default`, `object-wrapper`, or `bridge-mapping`.

Validate profile files before committing them:

```
cargo run -p gemstone-rs-cli -- profile validate examples/codegen/gemstone-
rs.codegen-profiles.json
cargo run -p gemstone-rs-cli -- profile check examples/codegen/gemstone-rs.codegen-
profiles.json
cargo run -p gemstone-rs-cli -- codegen explain-profile --json default examples/
codegen/gemstone-rs.codegen-profiles.json
```

Generate a starter config from a live stone:

```
cargo run -p gemstone-rs-cli -- codegen discover examples/codegen/discovered.codegen
Object
cargo run -p gemstone-rs --example codegen_discover_mapping
```

Run the generated wrapper example after checking the generated file into the repository:

```
cargo run -p gemstone-rs --example generated_wrapper_app
```

VS Code

Install the workbench from:

```
https://marketplace.visualstudio.com/items?itemName=unicompute.gemstone-rs-workbench
```

Use the GemStone RS sidebar to browse dictionaries, classes, protocols, and methods. The same sidebar exposes Codegen Discover, Preview, Diff, Check, Generate, profile-driven codegen actions, Generate Mapping Config, Preview BridgeRoot, List BridgeRoot Keys, Put BridgeRoot String, Put BridgeRoot Symbol, Put BridgeRoot SmallInt, Put BridgeRoot Bool, Remove BridgeRoot Key, Run Generated Mapping Example, Load Project Profiles, Save Project Profiles, Export Codegen Profile, Show Sample Project Profiles, Create Project Profiles, Validate Project Profiles, List Project Profiles, Show Project Profile, Resolve Project Profile, Check Project Profiles, and Open Docs actions.

Later Examples

These are useful, but should wait until the corresponding APIs are stable:

- a local explorer workflow with screenshots
- a full Axum or Actix workspace member wired into CI
- a richer class browser walkthrough with captured output

Cookbook

This cookbook is a collection of direct `gemstone-rs` recipes. Each recipe is small enough to copy into a Rust CLI, service, or maintenance tool.

Recipe 1: Open a Session and Evaluate Smalltalk

```
use gemstone_rs::{Config, Session};

let mut session = Session::login(Config::from_env()?)?;
println!("{:?}", session.eval("3 factorial")?);
```

Use this when you need the smallest live sanity check.

Recipe 2: Verify `3 + 4`

```
use gemstone_rs::{Config, Session, Value};

let mut session = Session::login(Config::from_env()?)?;
assert_eq!(session.eval("3 + 4")?, Value::SmallInt(7));
```

This is the test to keep in every live smoke lane.

Recipe 3: Write and Read `UserGlobals`

```
use gemstone_rs::{Config, Oop, Session};

let mut session = Session::login(Config::from_env()?)?;
let key = "GemStoneRsCookbook";
let value = session.new_string("stored from Rust")?;
session.global_put(key, value)?;

let stored = session.global_get(key)?;
println!("{}", session.fetch_string(stored)?);

session.global_put(key, Oop::NIL)?;
```

Recipe 4: Commit a Write Safely

```
session.transaction(|session| {
    let value = session.new_string("committed")?;
    session.global_put("GemStoneRsCommitted", value)
})?;
```

The closure commits only when it returns `Ok`.

Recipe 5: Abort on Error

```
use gemstone_rs::Error;

let result: gemstone_rs::Result<()> = session.transaction(|session| {
    let value = session.new_string("will abort")?;
    session.global_put("GemStoneRsAborted", value)?;
    Err(Error::IllegalOop {
        operation: "intentional abort",
    })
});
```

```
assert!(result.is_err());
```

Recipe 6: Convert Rust Values to OOPs

```
use gemstone_rs::Value;

let seven = session.value_to_oop(&Value::SmallInt(7))?;
let yes = session.bool_oop(true);
let nothing = session.nil_oop();
```

Recipe 7: Fetch a String

```
let string_oop = session.new_string("hello")?;
let text = session.fetch_string(string_oop)?;
assert_eq!(text, "hello");
```

Recipe 8: Send a Message and Keep the Raw OOP

```
let seven = session.smallint_oop(7);
let printed_oop = session.perform_oop(seven, "printString", &[])?;
println!("{}", session.fetch_string(printed_oop)?);
```

Recipe 9: Send a Message and Marshal the Result

```
let seven = session.smallint_oop(7);
let value = session.perform(seven, "class", &[])?;
println!("{value:?}");
```

Recipe 10: Retain a Long-Lived OOP

```
let text = session.new_string("retained")?;
let handle = session.retain_oop(text)?;
println!("retained OOP {}", handle.oop().raw());
handle.release()?;
```

The handle releases automatically on drop if `release()` is not called.

Recipe 10A: Store a Rust Payload Under BridgeRoot

```
use gemstone_rs::{BridgeValue, Config, Session};
use std::collections::BTreeMap;

let mut session = Session::login(Config::from_env()?)?;
let labels = BTreeMap::from([("source".to_string(), "cookbook".to_string())]);
let payload = BridgeValue::dictionary([
    ("name".to_string(), BridgeValue::from("Tariq")),
    ("amount".to_string(), BridgeValue::from(100_i64)),
    ("currency".to_string(), BridgeValue::from("GBP")),
    ("labels".to_string(), BridgeValue::from(labels)),
]);

let mut bridge_root = session.bridge_root()?;
bridge_root.put("MyTestDict", payload)?;
bridge_root.commit()?;
```

The default root is `UserGlobals at: #GemStoneRsBridgeRoot`.

Recipe 10B: Round-Trip a Typed BridgeRoot Map Field

```
use gemstone_rs::{
    BridgeDictionary, BridgeFieldWrite, BridgeKeyType, BridgeMapped, BridgeValue,
Config, Session,
};
use std::collections::BTreeMap;

#[derive(Debug, Eq, PartialEq)]
struct BookingDraft {
    name: String,
    labels: BTreeMap<String, String>,
}

impl BridgeMapped for BookingDraft {
    fn to_bridge_value(&self) -> BridgeValue {
        BridgeValue::dictionary([
            ("name".to_string(), BridgeValue::from(self.name.clone())),
            ("labels".to_string(),
BridgeFieldWrite::to_bridge_field_value(&self.labels)),
        ])
    }

    fn from_bridge_dictionary(dictionary: &mut BridgeDictionary<'_>) ->
gemstone_rs::Result<Self> {
        Ok(Self {
            name: dictionary.at_string("name")?,
            labels: dictionary.at_map("labels")?,
```

```
        })
    }
}

let mut session = Session::login(Config::from_env()?)?;
let draft = BookingDraft {
    name: "Tariq".to_string(),
    labels: BTreeMap::from([("source".to_string(), "cookbook".to_string())]),
};

let mut bridge_root = session.bridge_root()?;
bridge_root.put_mapped("CookbookBookingDraft", &draft)?;
let loaded: BookingDraft = bridge_root.get_mapped("CookbookBookingDraft")?;
assert_eq!(loaded.labels["source"], "cookbook");

bridge_root.put_string("CookbookStatus", "ready")?;
bridge_root.put_vec("CookbookTags", &["priority".to_string(), "demo".to_string()])?;
bridge_root.put_map("CookbookLabels", &draft.labels)?;
assert_eq!(bridge_root.get_string("CookbookStatus")?, "ready");
let tags: Vec<String> = bridge_root.get_vec("CookbookTags")?;
assert_eq!(tags, vec!["priority".to_string(), "demo".to_string()]);
let labels: BTreeMap<String, String> = bridge_root.get_map("CookbookLabels")?;
assert_eq!(labels["source"], "cookbook");

bridge_root.put_map_with_key_type("CookbookLabelsSymbol", BridgeKeyType::Symbol,
&draft.labels)?;
let symbol_labels: BTreeMap<String, String> =
    bridge_root.get_map_with_key_type("CookbookLabelsSymbol",
BridgeKeyType::Symbol)?;
assert_eq!(symbol_labels["source"], "cookbook");
```

Use map fields for string-keyed metadata or lookup tables. Use nested `BridgeMapped` structs when the related value has a stable domain shape.

Recipe 11: Browse Dictionaries

```
use gemstone_rs::browser::Browser;

let mut browser = Browser::new(&mut session);
for dictionary in browser.dictionaries()? {
    println!("{dictionary}");
}
```

Recipe 12: Browse a Class

```
use gemstone_rs::browser::{Browser, ALL_PROTOCOLS};

let mut browser = Browser::new(&mut session);
```

```
let protocols = browser.protocols("Object", false, "")?;
let methods = browser.methods("Object", ALL_PROTOCOLS, false, "")?;
let source = browser.source("Object", "printString", false, "")?;
```

Pass an empty dictionary to resolve through the active user's symbol list.

Recipe 13: Diagnose a New Machine

```
gemstone-rs doctor
gemstone-rs doctor --live
gemstone-rs doctor --json
```

The first command checks environment and GCI loading, including whether `libgcirpc` came from explicit config, `GS_LIB_PATH`, `GS_LIB`, or `GEMSTONE/lib`, plus the exact path or directory searched. The second command logs in and verifies that `3 + 4` returns `7`. The JSON form is useful in CI or editor tooling.

Recipe 14: Inspect an OOP From the CLI

```
gemstone-rs inspect oop 20
```

This prints the raw OOP, class OOP, and `printString`.

Recipe 15: Inspect BridgeRoot From the CLI

```
gemstone-rs bridge root
gemstone-rs bridge keys
gemstone-rs bridge get BookingDraft --symbol
gemstone-rs bridge inspect BookingDraft --symbol
gemstone-rs bridge put-string WorkbenchDraft "hello from Rust"
gemstone-rs bridge put-symbol WorkbenchState ready
gemstone-rs bridge put-smallint WorkbenchCount 7
gemstone-rs bridge put-bool WorkbenchReady true
gemstone-rs bridge remove WorkbenchDraft
```

Use this when object-mapping examples or generated mappings write payloads under `GemStoneRsBridgeRoot`. `bridge keys` is the quickest way to see what is currently stored before inspecting a specific payload. The `put-*` shortcuts and `bridge remove` are useful live-write smoke tests because they commit one explicit BridgeRoot change at a time. Use generic `bridge put --type ...` when you want the value type to be data-driven in a script.

Recipe 16: Preview Codegen

```
gemstone-rs codegen preview examples/codegen/gemstone-rs.codegen
```

Use preview before writing generated files.

Recipe 17: Check Generated Code in CI

```
gemstone-rs codegen check examples/codegen/gemstone-rs.codegen
```

The repository runs the same check through `make verify`.

Recipe 18: Generate Wrappers

```
gemstone-rs codegen generate examples/codegen/gemstone-rs.codegen
```

Generated files are meant to be checked in.

Recipe 19: Discover a Config From a Live Stone

```
gemstone-rs codegen discover examples/codegen/discovered.codegen Object
```

Use this to bootstrap a config, then edit the generated mapping down to the API you actually want.

Recipe 20: Run the Local Explorer

```
gemstone-rs-explorer --port 8787
```

Open:

```
http://127.0.0.1:8787/
```

The explorer starts read-only and loopback-only.

Recipe 21: Add a Local Explorer Auth Token

```
export GEMSTONE_RS_EXPLORER_TOKEN='replace-with-a-local-random-token'
gemstone-rs-explorer --port 8787 --auth-token-env GEMSTONE_RS_EXPLORER_TOKEN
curl -s 'http://127.0.0.1:8787/api/config?token=replace-with-a-local-random-token'
```

VS Code users can run `GemStone RS: Generate Explorer Auth Token`; the workbench stores the token in `gemstoneRs.explorerAuthToken`, launches the explorer with `--auth-token-env GEMSTONE_RS_EXPLORER_TOKEN`, and opens token-aware browser/webview URLs.

Recipe 22: Enable Explorer Eval Explicitly

```
gemstone-rs-explorer --port 8787 --allow-eval
curl -s 'http://127.0.0.1:8787/api/eval?source=3%20%2B%204'
```

Use this locally only.

Recipe 23: Use the VS Code Workbench With a Checkout

```
{
  "gemstoneRs.checkoutPath": "/path/to/gemstone-rs",
  "gemstoneRs.useCargo": true,
  "gemstoneRs.codegenConfig": "examples/codegen/gemstone-rs.codegen"
}
```

Then open the GemStone RS sidebar and use the Dictionaries, Codegen Config, and Explorer trees.

Recipe 24: Verify Published Artifacts

```
scripts/publish_verify.sh 0.2.2
```

The script checks crates.io versions, installs the CLI and explorer, runs `--help`, and checks the Marketplace version.

gemstone-py vs gemstone-rs

`gemstone-py` and `gemstone-rs` solve related problems at different maturity levels.

Use `gemstone-py` when you want the most complete Python experience today: Python sessions, async examples, web framework adapters, native acceleration, codegen, VS Code workbench integration, and a Python database explorer.

Use `gemstone-rs` when you want Rust services, CLIs, workers, or tooling to talk to GemStone/S directly without keeping Python in the process.

Install Paths

Goal	gemstone-py	gemstone-rs
Application library	`python -m pip install gemstone-py`	`cargo add gemstone-rs`
Native acceleration	`python -m pip install "gemstone-py[fast]"`	Built into the Rust GCI path once `libgcirpc` is available
Examples from source	`python -m pip install -e ".[examples]"`	`cargo run -p gemstone-rs --example quickstart`
CLI tooling	Python modules and examples	`cargo install gemstone-rs-cli`
Local explorer	`python-gemstone-database-explorer`	`cargo install gemstone-rs-explorer`
VS Code	`gemstone-py Workbench`	`gemstone-rs Workbench`

Both projects use the same GemStone-style environment:

```
export GS_LIB=/opt/gemstone/product/lib
export GS_STONE=gs64stone
export GS_USERNAME=DataCurator
export GS_PASSWORD=your-password
```

`GS_STONE_NAME` is accepted as a stone-name alias where the clients support it. Use `GS_LIB_PATH` when you want to point directly at a specific `libgcirpc` file.

Best Fit

Use case	Better choice	Why
Python web app	gemstone-py	FastAPI and Litestar examples are already first-class.
Python scripts and notebooks	gemstone-py	Lower friction for Python teams.
Rust service or worker	gemstone-rs	Native Rust API, Rust errors, Rust ownership, Cargo distribution.
Rust CLI tooling	gemstone-rs	CLI and explorer can run without Python.
VS Code database/ codegen workflow	gemstone-py today, gemstone-rs later	gemstone-py has the more mature explorer; gemstone-rs has the cleaner Rust backend direction.
Shared low-level bridge	gemstone-rs	The Rust API is the right place to isolate GCI safety and ownership.

Maturity Matrix

Capability	gemstone-py	gemstone-rs
Stable install path	Mature: PyPI and TestPyPI	Early: crates.io workflow and verification added
Native bridge	PyO3 extension path	Rust GCI crate and safe API
Sync API	Mature	Initial safe API
Async API	Mature enough for examples and tests	Not yet a core feature
Web frameworks	FastAPI, Litestar, Django examples	Axum service sketch added; full app example still planned
Codegen	Python wrapper workflow	Rust wrapper workflow with preview/diff/check/ generate
Browser API	Used by database explorer	CLI/explorer API for dictionaries/classes/methods/ source

Capability	gemstone-py	gemstone-rs
Local explorer	More mature Python app	Minimal Rust explorer proving the API
VS Code extension	More complete product flow	Thin command/workbench layer over Rust CLI and explorer
Live tests	Broader coverage	Growing smoke suite for login/eval/perform/globals/transactions/codegen
Release automation	More complete	Added crates/VSIX/GitHub verification path
Docs	Broader and polished	Catching up with guide, cookbook, article, comparison, PDFs

Feature Matrix

Feature	gemstone-py status	gemstone-rs status
Login/logout	Supported	Supported
`3 + 4 == 7` smoke	Supported	Supported
Global put/get	Supported	Supported
String round-trip	Supported	Supported
`perform` and `perform_oop`	Supported	Supported
Commit/abort	Supported	Supported
OOP handles/export set	Supported	Supported
Browser dictionaries/classes/protocols/methods/source	Supported	Supported through `Browser`, CLI, and explorer
Generated wrappers	Supported	Supported for selected classes/methods
Codegen diff/check/generate	Supported	Supported
Live codegen discovery	Supported	Supported through CLI/API
FastAPI demo	Supported	Not applicable

Feature	gemstone-py status	gemstone-rs status
Litestar demo	Supported	Not applicable
Axum/Actix demo	Not applicable	Axum sketch documented; full app example still planned
Database explorer	Mature Python app	Rust explorer has browser UI and local API; still less polished
VS Code sidebar workflow	More complete	Rust workbench has sidebar commands and embedded explorer webview

How They Should Work Together

The best long-term shape is not competition between the projects. It is a clean boundary:

- `gemstone-rs` owns the direct Rust/GCI interface and safe Rust API.
- `gemstone-py-native` can become a thin PyO3 layer over the Rust core.
- `gemstone-py` remains the Python API, examples, and Python web integration.
- The Python and Rust explorers can share concepts and eventually converge on common codegen workflows.

That gives Python users the friendliest path and Rust users a direct path, while keeping unsafe GCI behavior isolated in one Rust layer.

Recommendation

For production Python projects, start with `gemstone-py`.

For Rust-native services, CLIs, workers, and tooling, start with `gemstone-rs`, but treat it as an early API that should be validated against a live stone in your environment.

For VS Code and visual codegen work, use the existing Python explorer as the product reference and keep moving `gemstone-rs-explorer` toward the same level of capability.

BridgeRoot and Object Mapping

`gemstone-rs` now has a first object-mapping layer over the lower-level `Oop` API.

This is intentionally smaller than MagLev/GBS object mapping. It gives Rust code a bridge-root dictionary, typed Rust payload mapping, nested read-back, and generated mapping support while keeping direct OOP access available.

What Is Supported

The initial mapping layer supports:

- `Session::bridge_root()`
- `Session::bridge_root_named(name)`
- `BridgeRoot::put`
- `BridgeRoot::put_mapped`
- `BridgeRoot::put_mapped_with_key_type`
- `BridgeRoot::put_field`
- `BridgeRoot::put_field_with_key_type`
- `BridgeRoot::put_string`
- `BridgeRoot::put_string_with_key_type`
- `BridgeRoot::put_symbol`
- `BridgeRoot::put_symbol_with_key_type`
- `BridgeRoot::put_smallint`
- `BridgeRoot::put_smallint_with_key_type`
- `BridgeRoot::put_bool`
- `BridgeRoot::put_bool_with_key_type`
- `BridgeRoot::put_vec`
- `BridgeRoot::put_vec_with_key_type`
- `BridgeRoot::put_map`
- `BridgeRoot::put_map_with_key_type`
- `BridgeRoot::put_optional`
- `BridgeRoot::put_optional_with_key_type`
- `BridgeRoot::get_oop`
- `BridgeRoot::get_value`
- `BridgeRoot::get_field`
- `BridgeRoot::get_field_with_key_type`
- `BridgeRoot::get_vec`
- `BridgeRoot::get_vec_with_key_type`
- `BridgeRoot::get_map`
- `BridgeRoot::get_map_with_key_type`
- `BridgeRoot::get_optional`
- `BridgeRoot::get_optional_with_key_type`
- `BridgeRoot::get_string`
- `BridgeRoot::get_string_with_key_type`
- `BridgeRoot::get_smallint`
- `BridgeRoot::get_smallint_with_key_type`
- `BridgeRoot::get_bool`
- `BridgeRoot::get_bool_with_key_type`

- `BridgeRoot::get_dictionary`
- `BridgeRoot::get_dictionary_with_key_type`
- `BridgeRoot::get_mapped`
- `BridgeRoot::get_mapped_with_key_type`
- `BridgeRoot::remove`
- `BridgeRoot::remove_with_key_type`
- `BridgeRoot::commit`
- `BridgeRoot::commit_with_retry`
- `BridgeRoot::transaction`
- `BridgeRoot::keys`
- `BridgeRoot::contains_key`
- `BridgeDictionary::at_oop`
- `BridgeDictionary::at_value`
- `BridgeDictionary::at_string`
- `BridgeDictionary::at_smallint`
- `BridgeDictionary::at_bool`
- `BridgeDictionary::at_dictionary`
- `BridgeDictionary::at_field`
- `BridgeDictionary::at_mapped`
- `BridgeDictionary::at_vec`
- `BridgeDictionary::at_map`
- `BridgeDictionary::at_optional`
- `BridgeDictionary::put_field`
- `BridgeDictionary::put_string`
- `BridgeDictionary::put_symbol`
- `BridgeDictionary::put_smallint`
- `BridgeDictionary::put_bool`
- `BridgeDictionary::put_vec`
- `BridgeDictionary::put_map`
- `BridgeDictionary::put_optional`
- `BridgeDictionary::contains_key`
- `BridgeDictionary::keys`
- `BridgeKey`
- `BridgeKeyType::String`
- `BridgeKeyType::Symbol`
- `BridgeMapped`
- `#[derive(BridgeMapped)]`
- `BridgeFieldRead`
- `BridgeFieldWrite`
- `BridgeValue::Nil`
- `BridgeValue::Bool`
- `BridgeValue::SmallInt`
- `BridgeValue::String`
- `BridgeValue::Symbol`
- `BridgeValue::Oop`

- `BridgeValue::Dictionary`
- `BridgeValue::KeyedDictionary`
- `BridgeValue::Array`
- session-local OOP identity ids with `Session::identity_for_oop`

The default root is a GemStone `Dictionary` stored in `UserGlobals` under `#GemStoneRsBridgeRoot`.

Rust Example

```
use gemstone_rs::{BridgeValue, Config, Session};

fn main() -> gemstone_rs::Result<()> {
    let mut session = Session::login(Config::from_env()?)?;

    let payload = BridgeValue::dictionary([
        ("name".to_string(), BridgeValue::from("Tariq")),
        ("amount".to_string(), BridgeValue::from(100_i64)),
        ("currency".to_string(), BridgeValue::from("GBP")),
    ]);

    let mut bridge_root = session.bridge_root()?;
    let payload_oop = bridge_root.put("MyTestDict", payload)?;
    let stored = bridge_root.get_oop("MyTestDict")?;
    assert_eq!(payload_oop, stored);

    bridge_root.commit()?;
    Ok(())
}
```

Run the example:

```
cargo run -p gemstone-rs --example bridge_root_mapping
```

Expected output includes:

```
bridge root: GemStoneRsBridgeRoot
MyTestDict OOP: <number>
```

Typed Rust Struct Mapping

For application code, implement `BridgeMapped` on a plain Rust type. The trait keeps the mapping explicit and reviewable while removing repeated dictionary field reads from application code.

```
use gemstone_rs::{
    BridgeDictionary, BridgeFieldWrite, BridgeKeyType, BridgeMapped, BridgeValue,
Config, Session,
};
use std::collections::BTreeMap;

#[derive(Debug, Eq, PartialEq)]
struct BookingDraft {
    name: String,
    amount: i64,
    currency: String,
    labels: BTreeMap<String, String>,
}

impl BridgeMapped for BookingDraft {
    fn to_bridge_value(&self) -> BridgeValue {
        BridgeValue::dictionary([
            ("name".to_string(), BridgeValue::from(self.name.clone())),
            ("amount".to_string(), BridgeValue::from(self.amount)),
            ("currency".to_string(), BridgeValue::from(self.currency.clone())),
            ("labels".to_string(),
BridgeFieldWrite::to_bridge_field_value(&self.labels)),
        ])
    }

    fn from_bridge_dictionary(dictionary: &mut BridgeDictionary<'_>) ->
gemstone_rs::Result<Self> {
        Ok(Self {
            name: dictionary.at_string("name")?,
            amount: dictionary.at_smallint("amount")?,
            currency: dictionary.at_string("currency")?,
            labels: dictionary.at_map("labels")?,
        })
    }
}

fn main() -> gemstone_rs::Result<()> {
    let mut session = Session::login(Config::from_env()?)?;
    let mut bridge_root = session.bridge_root()?;

    let draft = BookingDraft {
        name: "Tariq".to_string(),
        amount: 100,
        currency: "GBP".to_string(),
        labels: BTreeMap::from([("source".to_string(), "manual".to_string())]),
    };

    bridge_root.put_mapped("MyTestDict", &draft)?;
    let loaded: BookingDraft = bridge_root.get_mapped("MyTestDict")?;
    assert_eq!(loaded, draft);

    bridge_root.put_string("MyTestStatus", "ready")?;
    bridge_root.put_smallint("MyTestAmount", draft.amount)?;
    bridge_root.put_bool("MyTestApproved", true)?;
```

```
    bridge_root.put_vec("MyTestTags", &["priority".to_string(),
"demo".to_string()])?;
    bridge_root.put_optional("MyTestNote", &Some("front desk".to_string()))?;
    bridge_root.put_map("MyTestLabels", &draft.labels)?;

    assert_eq!(bridge_root.get_string("MyTestStatus")?, "ready");
    assert_eq!(bridge_root.get_smallint("MyTestAmount")?, draft.amount);
    assert!(bridge_root.get_bool("MyTestApproved")?);
    let tags: Vec<String> = bridge_root.get_vec("MyTestTags")?;
    assert_eq!(tags, vec!["priority".to_string(), "demo".to_string()]);
    let note: Option<String> = bridge_root.get_optional("MyTestNote")?;
    assert_eq!(note, Some("front desk".to_string()));
    let labels: BTreeMap<String, String> = bridge_root.get_map("MyTestLabels")?;
    assert_eq!(labels, draft.labels);

    bridge_root.put_map_with_key_type(
        "MyTestLabelsSymbol",
        BridgeKeyType::Symbol,
        &draft.labels,
    )?;
    let symbol_labels: BTreeMap<String, String> =
        bridge_root.get_map_with_key_type("MyTestLabelsSymbol",
BridgeKeyType::Symbol)?;
    assert_eq!(symbol_labels, draft.labels);

    bridge_root.commit()?;
    Ok(())
}
```

The checked-in example uses this pattern:

```
cargo run -p gemstone-rs --example bridge_root_mapping
```

Expected output includes:

```
bridge root: GemStoneRsBridgeRoot
MyTestDict OOP: <number>
loaded payload: BookingDraft { name: "Tariq", amount: 100, currency: "GBP", labels:
{"source": "manual"} }
loaded status: ready
loaded amount: 100
loaded approved: true
loaded tags: ["priority", "demo"]
loaded note: Some("front desk")
loaded labels: {"source": "manual"}
loaded symbol labels: {"source": "manual"}
```

Derive-Based Mapping

For normal Rust structs, prefer `#[derive(BridgeMapped)]`. The derive writes a `BridgeMapped` implementation that stores fields in a GemStone dictionary and reads them back into Rust.

```
use gemstone_rs::{BridgeMapped, Config, Session};
use std::collections::BTreeMap;

#[derive(Clone, Debug, Eq, PartialEq, BridgeMapped)]
struct CustomerDraft {
    #[bridge(key_type = "Symbol")]
    name: String,
}

#[derive(Clone, Debug, Eq, PartialEq, BridgeMapped)]
struct BookingDraft {
    #[bridge(key = "amount", key_type = "Symbol")]
    amount: i64,
    customer: CustomerDraft,
    tags: Vec<String>,
    labels: BTreeMap<String, String>,
    note: Option<String>,
}

fn main() -> gemstone_rs::Result<()> {
    let mut session = Session::login(Config::from_env()?)?;
    let mut bridge_root = session.bridge_root()?;

    let draft = BookingDraft {
        amount: 100,
        customer: CustomerDraft {
            name: "Tariq".to_string(),
        },
        tags: vec!["priority".to_string(), "demo".to_string()],
        labels: BTreeMap::from([("source".to_string(), "derive".to_string())]),
        note: None,
    };

    bridge_root.put_mapped("DerivedBookingDraft", &draft)?;
    let loaded: BookingDraft = bridge_root.get_mapped("DerivedBookingDraft")?;
    assert_eq!(loaded, draft);

    bridge_root.commit()?;
    Ok(())
}
```

Run the checked-in live example:

```
cargo run -p gemstone-rs --example derive_mapping
```

Expected output includes:

```
derived mapped payload: BookingDraft { amount: 100, customer: CustomerDraft { name:
"Tariq" }, tags: ["priority", "demo"], labels: {"source": "derive"}, note: None }
bridge root identity: <number>
```

Nested Read-Back

Nested mapped structs and arrays round-trip through the same bridge dictionary format:

Rust field	GemStone storage	Read-back helper
`String`	`String`	`BridgeFieldRead` / `at_string`
`i64`	`SmallInteger`	`BridgeFieldRead` / `at_smallint`
`bool`	`true` / `false`	`BridgeFieldRead` / `at_bool`
`Oop`	raw object reference	`BridgeFieldRead` / `at_oop`
another `BridgeMapped` struct	nested `Dictionary`	`BridgeDictionary::at_mapped`
`Vec<T>`	`Array`	`BridgeDictionary::at_vec`
`BTreeMap<String, T>`	string-keyed `Dictionary`	`BridgeDictionary::at_map`
`Option<T>`	value, `nil`, or missing key	`BridgeFieldRead`

This lets a Rust payload keep normal nested shape:

```
#[derive(Clone, Debug, Eq, PartialEq, BridgeMapped)]
struct BookingDraft {
    customer: CustomerDraft,
    tags: Vec<String>,
    labels: BTreeMap<String, String>,
    note: Option<String>,
}
```

Optional fields are useful when a BridgeRoot dictionary evolves over time. `None` writes as GemStone `nil`; read-back returns `None` when the key is missing or when the stored value is `nil`. `Some(value)` reads through the inner field type, so `Option<CustomerDraft>` still reports nested mapping errors with the full field path.

When read-back fails, mapping errors include field context:

```
field amount expected GemStone value type SmallInt, got Oop 1234
field booking.customer.name expected GemStone value type String, got OOP 1234
```

```
field tags[2] expected GemStone value type String, got OOP 1234
field labels["source"] expected GemStone value type String, got OOP 1234
```

Nested mapped read-back preserves the full field path, and array read-back reports the failing element index. Both are important when generated mappings read back nested payloads from a live stone. Lookup failures are also wrapped with the current path, so missing keys and invalid nested arrays point at `booking.items` or `booking.items[2]` instead of only returning a generic GemStone lookup error.

Key Policy

GemStone dictionaries often use either string keys or symbol keys. The mapping layer makes that explicit:

```
#[derive(Clone, Debug, Eq, PartialEq, BridgeMapped)]
struct BookingDraft {
    #[bridge(key = "amount", key_type = "Symbol")]
    amount: i64,
}
```

Manual code can use the same policy:

```
use gemstone_rs::BridgeKeyType;

bridge_root.put_with_key_type("BookingDraft", BridgeKeyType::Symbol, "stored")?;
let oop = bridge_root.get_oop_with_key_type("BookingDraft", BridgeKeyType::Symbol)?;
```

The typed helpers have the same key-policy variants:

```
bridge_root.put_field_with_key_type("BookingLabels", BridgeKeyType::Symbol,
&draft.labels)?;
let labels: BTreeMap<String, String> =
    bridge_root.get_map_with_key_type("BookingLabels", BridgeKeyType::Symbol)?;
```

Use string keys when interoperating with JSON-like payloads and symbol keys when you want Smalltalk-style dictionary access.

Transactions and Identity

`BridgeRoot::transaction` commits on success and aborts on error:

```
bridge_root.transaction(|root| {
    root.put_mapped("BookingDraft", &draft)?;
```

```
    let loaded: BookingDraft = root.get_mapped("BookingDraft")?;
    assert_eq!(loaded, draft);
    Ok(())
})?;
```

For conflict-prone commits, use a bounded retry:

```
bridge_root.put_mapped("BookingDraft", &draft)?;
bridge_root.commit_with_retry(2)?;
```

`Session` also tracks a session-local identity id for OOPs it sees:

```
let oop = bridge_root.get_oop("BookingDraft")?;
let identity = bridge_root.identity_id();
println!("bridge root identity={identity}, payload oop={}", oop.raw());
```

That is not transparent object persistence. It is a stable in-session cache key for wrappers, inspectors, and explorer views.

Relationship-Shaped Payloads

Use nested structs and vectors when the Rust boundary needs a relationship-like shape but you still want explicit persistence. This keeps the GemStone side a plain dictionary graph and keeps the Rust side strongly typed:

```
use gemstone_rs::BridgeMapped;
use std::collections::BTreeMap;

#[derive(Clone, Debug, Eq, PartialEq, BridgeMapped)]
struct CustomerDraft {
    name: String,
    email: String,
}

#[derive(Clone, Debug, Eq, PartialEq, BridgeMapped)]
struct LineItemDraft {
    sku: String,
    quantity: i64,
}

#[derive(Clone, Debug, Eq, PartialEq, BridgeMapped)]
struct BookingDraft {
    customer: CustomerDraft,
    items: Vec<LineItemDraft>,
    tags: Vec<String>,
    labels: BTreeMap<String, String>,
```

```
    note: Option<String>,
}
```

A failed nested read reports the path that a user would naturally inspect:

```
booking.items[2].customer.name expected GemStone value type String, got OOP 1234
```

This is the practical middle ground between raw OOPs and transparent object persistence: relationships are visible in config and generated Rust source, and the low-level OOP API remains available for special cases.

Comparison With MagLev/GBS

The MagLev branch example uses:

```
session bridgeRoot at: #MyTestDict put: payload.
session commitTransactionOrSignalConflict
```

The closest current `gemstone-rs` shape is:

```
let mut bridge_root = session.bridge_root()?;
bridge_root.put("MyTestDict", payload)?;
bridge_root.commit()?;
```

For typed Rust structs:

```
bridge_root.put_mapped("MyTestDict", &draft)?;
let loaded: BookingDraft = bridge_root.get_mapped("MyTestDict")?;
```

This is still explicit mapping, not transparent persistence. The benefit is that Rust teams can review every field mapping and still keep direct OOP access available when they need it.

Codegen Support

`gemstone-rs` codegen can emit `BridgeMapped` structs from config:

```
mapped = BookingDraft | doc=A typed Rust payload stored under BridgeRoot.
field = BookingDraft.name | type=String | key=name
field = BookingDraft.amount | type=SmallInt | key=amount | key_type=Symbol
field = BookingDraft.currency | type=String | key=currency
field = BookingDraft.tags | type=Vec<String> | key=tags
```

```
field = BookingDraft.labels | type=BTreeMap<String, String> | key=labels
field = BookingDraft.note | type=Option<String> | key=note
```

Generate a mapping config proposal from a live GemStone class:

```
gemstone-rs codegen discover-mapping gemstone-rs.codegen BookingDraft Object
```

For quick CLI inspection of the bridge root itself:

```
gemstone-rs bridge root
gemstone-rs bridge keys
gemstone-rs bridge get BookingDraft --symbol
gemstone-rs bridge inspect BookingDraft --symbol
gemstone-rs bridge put-string WorkbenchDraft "hello from Rust"
gemstone-rs bridge put-symbol WorkbenchState ready
gemstone-rs bridge put-smallint WorkbenchCount 7
gemstone-rs bridge put-bool WorkbenchReady true
gemstone-rs bridge remove WorkbenchDraft
gemstone-rs bridge sample-config BookingDraft
```

Use `--root OtherBridgeRoot` when you intentionally use a non-default root dictionary, and `--key-type String|Symbol` when you want the key policy to be spelled out in scripts. Equals-style options such as `--root=OtherBridgeRoot`, `--key-type=Symbol`, and `--type=SmallInt` are accepted for CI scripts. The generic `bridge put --type String|Symbol|SmallInt|Bool` form is still available when the value type comes from script data.

`bridge keys` lists each root key with its key OOP, class OOP, `printString`, and session-local identity id.

Run the live discovery example:

```
cargo run -p gemstone-rs --example codegen_discover_mapping
```

Run the generated mapping example:

```
cargo run -p gemstone-rs --example generated_mapping_app
```

This is the recommended path for repeated mappings because the generated source is checked in and reviewed like any other Rust code.

Explorer and VS Code Support

The local explorer exposes BridgeRoot-oriented endpoints:

```
curl -s http://127.0.0.1:8787/api/bridge/root
curl -s http://127.0.0.1:8787/api/bridge/keys
curl -s 'http://127.0.0.1:8787/api/bridge/get?key=BookingDraft'
curl -s 'http://127.0.0.1:8787/api/bridge/put?key=WorkbenchDraft&value=hello'
curl -s 'http://127.0.0.1:8787/api/bridge/remove?key=WorkbenchDraft'
curl -s 'http://127.0.0.1:8787/api/bridge/mapping-config?mapped=BookingDraft'
curl -s 'http://127.0.0.1:8787/api/codegen/discover-mapping?
mapped=BookingDraft&class=Object'
```

The write endpoints require `gemstone-rs-explorer --allow-write`. Without that flag they return HTTP 403, matching the explorer's read-only default.

The VS Code workbench adds the same object-mapping workflow:

- `GemStone RS: Generate Mapping Config`
- `GemStone RS: Preview BridgeRoot`
- `GemStone RS: List BridgeRoot Keys`
- `GemStone RS: Put BridgeRoot String`
- `GemStone RS: Put BridgeRoot Symbol`
- `GemStone RS: Put BridgeRoot SmallInt`
- `GemStone RS: Put BridgeRoot Bool`
- `GemStone RS: Remove BridgeRoot Key`
- `GemStone RS: Run Generated Mapping Example`

The sidebar also shows these actions under `Codegen Config`.

Current Boundaries

The mapping layer is explicit, not transparent persistence. It does not yet make every GemStone object look like a native Rust object automatically. The current design is deliberately conservative:

- `Oop` remains visible and available.
- `Session` is still non-`Send` and non-`Sync`.
- writes happen only through explicit `put`, `put_mapped`, or generated code.
- the identity map is session-local and does not replace GemStone identity.

Rust Codegen

`gemstone-rs` codegen creates small Rust wrapper structs around GemStone OOPs. The goal is to move repeated selector strings into checked-in generated code while keeping the mapping reviewable.

Install

```
cargo install gemstone-rs-cli
gemstone-rs codegen --help
```

From a checkout:

```
cargo run -p gemstone-rs-cli -- codegen preview examples/codegen/gemstone-rs.codegen
cargo run -p gemstone-rs-cli -- codegen explain examples/codegen/gemstone-rs.codegen
cargo run -p gemstone-rs-cli -- codegen explain --json examples/codegen/gemstone-
rs.codegen
cargo run -p gemstone-rs-cli -- codegen check-profile default examples/codegen/
gemstone-rs.codegen-profiles.json
cargo run -p gemstone-rs-cli -- codegen explain-profile --json default examples/
codegen/gemstone-rs.codegen-profiles.json
```

Config Format

The config is intentionally line-oriented:

```
output = examples/codegen/generated/gemstone_wrappers.rs
class = Object
method = Object>>printString | return=String | doc=Return the receiver printString.
method = Object>>class
```

Use `Dictionary:ClassName` when a class should resolve from a specific dictionary:

```
class = UserGlobals:OkzBooking
method = UserGlobals:OkzBooking>>findById: | args=id | return=Oop | doc=Find a
booking by id.
```

Class-side references use `class`:

```
class = UserGlobals:OkzBooking class
method = UserGlobals:OkzBooking class>>findById: | args=id | return=Oop
```

Method Metadata

Option	Example	Purpose
`args`	`args=id,user`	Names generated Rust function arguments.

Option	Example	Purpose
`return`	`return=String`	Generates a typed return helper instead of `Value`.
`doc`	`doc=Find by id.`	Writes a Rust doc comment above the generated method.

When `args` is omitted, codegen now infers useful argument names from selector keywords. For example `at:put:` becomes `at, put`, and `withCustomer:amount:` becomes `customer, amount`. Provide explicit `args=...` when the selector keywords are not good Rust argument names.

Generated wrapper files include a small `#[cfg(test)]` surface-name test stub, so downstream crates can keep generated wrapper names visible to Rust test tooling. Use `codegen explain` before generating when you want a readable summary of the output file, test stub, wrapper classes, selectors, argument names, return helpers, mapped structs, and BridgeRoot field mappings:

```
gemstone-rs codegen explain examples/codegen/gemstone-rs.codegen
gemstone-rs codegen explain --json examples/codegen/gemstone-rs.codegen
gemstone-rs --env-file .env.gemstone-rs codegen check examples/codegen/gemstone-
rs.codegen
```

The JSON form is intended for editor and explorer integrations that want to render the output path, generated test stubs, class wrappers, selector arguments, return helpers, and mapped fields as structured data.

Machine-readable schema files are committed for tooling:

```
schemas/gemstone-rs.codegen.schema.json
schemas/gemstone-rs.codegen-explain.schema.json
schemas/gemstone-rs.codegen-profiles.schema.json
schemas/gemstone-rs.profile-check.schema.json
```

`gemstone-rs.codegen` remains the line-oriented CLI format. The config schema describes an equivalent structured JSON model for editor panels and generated summaries, while the explain schema matches `codegen explain --json`. The profile check schema matches `profile check --json` and the explorer `/api/codegen/profiles/check` endpoint.

Supported return types:

Config value	Rust return
`Value`	`gemstone_rs::Value`
`String`	`String`

Config value	Rust return
`SmallInt`	`i64`
`Bool`	`bool`
`Oop`	`gemstone_rs::Oop`

Mapped Structs

Codegen can also emit typed `BridgeMapped` structs for values stored under `BridgeRoot`:

```
mapped = BookingDraft | doc=A typed Rust payload stored under BridgeRoot.
field = BookingDraft.name | type=String | key=name
field = BookingDraft.amount | type=SmallInt | key=amount | key_type=Symbol
field = BookingDraft.currency | type=String | key=currency
field = BookingDraft.tags | type=Vec<String> | key=tags
field = BookingDraft.labels | type=BTreeMap<String, String> | key=labels |
doc=String-keyed labels.
field = BookingDraft.note | type=Option<String> | key=note | doc=Optional note.
```

Supported field types are `String`, `SmallInt`, `Bool`, `Oop`, `Mapped<OtherStruct>`, `Vec<T>`, `BTreeMap<String, T>`, and `Option<T>`. `Map<String, T>` is accepted as a shorter alias, and `Dictionary<T>` is an alias for `BTreeMap<String, T>`. Map fields store string-keyed relationship metadata as a GemStone `Dictionary`. Optional fields write `None` as GemStone `nil`; read-back returns `None` when the key is missing or when the stored value is `nil`.

Field options:

Option	Example	Purpose
`type`	`type=Option<String>`	Rust/GemStone field conversion.
`key`	`key=amount`	GemStone dictionary key.
`key_type`	`key_type=Symbol`	Use string keys or symbol keys explicitly.
`doc`	`doc=Booking amount.`	Writes a Rust doc comment above the field.

Commands

Create a starter config:

```
gemstone-rs codegen init gemstone-rs.codegen
```

Generate a config from a live stone:

```
gemstone-rs codegen discover gemstone-rs.codegen Object
gemstone-rs codegen discover gemstone-rs.codegen UserGlobals:OkzBooking
gemstone-rs codegen discover-mapping gemstone-rs.codegen BookingDraft Object
```

From a checkout:

```
cargo run -p gemstone-rs --example codegen_discover
cargo run -p gemstone-rs --example codegen_discover_mapping
```

Preview without writing:

```
gemstone-rs codegen preview gemstone-rs.codegen
```

Show a generated diff:

```
gemstone-rs codegen diff gemstone-rs.codegen
```

Check freshness in CI:

```
gemstone-rs codegen check gemstone-rs.codegen
cargo test --manifest-path examples/codegen-wrapper-check/Cargo.toml
```

Write generated wrappers:

```
gemstone-rs codegen generate gemstone-rs.codegen
```

Explorer Profiles

The local explorer can save repeatable codegen workflows as profiles. A profile captures:

- codegen root
- config path
- mapped Rust type
- GemStone class

Use the browser UI to save a local profile, export a single profile as JSON, or load a project-level profile file. The repository includes a sample:

```
examples/codegen/gemstone-rs.codegen-profiles.json
```

Validate it from CI or a terminal:

```
gemstone-rs profile sample
gemstone-rs profile init gemstone-rs.codegen-profiles.json
gemstone-rs profile validate gemstone-rs.codegen-profiles.json
gemstone-rs profile validate --json gemstone-rs.codegen-profiles.json
gemstone-rs profile list gemstone-rs.codegen-profiles.json
gemstone-rs profile show default gemstone-rs.codegen-profiles.json
gemstone-rs profile resolve default gemstone-rs.codegen-profiles.json
gemstone-rs profile check gemstone-rs.codegen-profiles.json
gemstone-rs profile check --json gemstone-rs.codegen-profiles.json
gemstone-rs codegen preview-profile default gemstone-rs.codegen-profiles.json
gemstone-rs codegen diff-profile default gemstone-rs.codegen-profiles.json
gemstone-rs codegen check-profile default gemstone-rs.codegen-profiles.json
gemstone-rs codegen explain-profile --json default gemstone-rs.codegen-profiles.json
gemstone-rs codegen generate-profile default gemstone-rs.codegen-profiles.json
```

See Codegen Profile Schema and `schemas/gemstone-rs.codegen-profiles.schema.json` for editor validation.

The project profile file uses this shape:

```
{"kind":"gemstone-rs-explorer-codegen-profiles","version":1,"profiles":
[{"name":"default","config":"examples/codegen/gemstone-
rs.codegen","root":"","mapped":"BookingDraft","className":"Object"}]}
```

Start the explorer with a project root:

```
gemstone-rs-explorer --port 8787 --codegen-root /path/to/gemstone-rs
```

Profile writes are disabled unless the explorer was started with `--allow-write`. The server rejects path traversal in `config=` and `profile_file=` after URL decoding, rejects traversal in `root=`, keeps writes under the configured codegen root by default, and requires `--allow-absolute-write-paths` before absolute write targets are accepted. Project profile saves reject unsupported top-level fields, unsupported profile fields, missing or duplicate profile names, and non-string profile values.

Generated Shape

For:

```
method = Object>>printString | return=String | doc=Return the receiver printString.
```

The generated wrapper includes:

```
pub fn print_string(&mut self) -> Result<String> {
    let value = self.session.perform(self.oop, "printString", &[])?;
    match value {
        Value::String(value) => Ok(value),
        Value::Oop(oop) => self.session.fetch_string(oop),
        other => Err(Error::UnexpectedType {
            expected: "String",
            actual: format!("{other:?}"),
        }),
    }
}
```

Generated files are meant to be checked in. That makes diffs, reviews, and editor indexing predictable.

Generated Wrapper App

The repository includes a small live example that imports the checked-in generated wrapper and calls `Object>>printString` on a small integer:

```
cargo run -p gemstone-rs --example generated_wrapper_app
```

Expected output:

```
generated wrapper printString: 7
```

The example uses the generated `Object::from_oop` constructor:

```
#[path = "../../../examples/codegen/generated/gemstone_wrappers.rs"]
mod gemstone_wrappers;

use gemstone_rs::{Config, Session};

fn main() -> gemstone_rs::Result<()> {
    let mut session = Session::login(Config::from_env()?)?;
    let oop = session.smallint_oop(7);
    let mut object = gemstone_wrappers::Object::from_oop(&mut session, oop);
    println!("{}", object.print_string()?);
    Ok(())
}
```

Generated Object Mapping

The generated file can also include Rust data structs that implement `BridgeMapped`. Run:

```
cargo run -p gemstone-rs --example generated_mapping_app
```

Expected output:

```
generated mapped payload: BookingDraft { name: "Tariq", amount: 100, currency: "GBP",
tags: ["priority", "demo"], labels: {"source": "generated"}, note: Some("window
seat") }
```

The config-driven mapping emits a struct like:

```
#[derive(Clone, Debug, Eq, PartialEq)]
pub struct BookingDraft {
    pub name: String,
    pub amount: i64,
    pub currency: String,
    pub tags: Vec<String>,
    /// String-keyed labels.
    pub labels: BTreeMap<String, String>,
    /// Optional note.
    pub note: Option<String>,
}
```

and an implementation of `BridgeMapped` so it works with:

```
bridge_root.put_mapped("GeneratedBookingDraft", &draft)?;
let loaded: BookingDraft = bridge_root.get_mapped("GeneratedBookingDraft")?;
```

VS Code Workflow

The `gemstone-rs Workbench` extension exposes the same flow in the GemStone RS sidebar:

- Discover from Live Stone
- Generate Mapping Config
- Preview BridgeRoot
- Run Generated Mapping Example
- Preview Wrappers
- Diff Generated Output
- Check Freshness
- Generate Wrappers
- Load Project Profiles

- Save Project Profiles
- Export Codegen Profile
- Validate Project Profiles
- Check Project Profiles
- Open Codegen Docs

`Generate Wrappers` shows the diff first and asks before writing when output would change.

Codegen Profile Schema

`gemstone-rs` project profile files make explorer and VS Code codegen workflows repeatable. The default file name is:

```
gemstone-rs.codegen-profiles.json
```

This repository includes a sample:

```
examples/codegen/gemstone-rs.codegen-profiles.json
```

The JSON Schema is checked in at:

```
schemas/gemstone-rs.codegen-profiles.schema.json
```

Validate the sample from a checkout:

```
cargo run -p gemstone-rs-cli -- profile sample
cargo run -p gemstone-rs-cli -- profile init gemstone-rs.codegen-profiles.json
cargo run -p gemstone-rs-cli -- profile validate examples/codegen/gemstone-
rs.codegen-profiles.json
cargo run -p gemstone-rs-cli -- profile validate --json examples/codegen/gemstone-
rs.codegen-profiles.json
cargo run -p gemstone-rs-cli -- profile list examples/codegen/gemstone-rs.codegen-
profiles.json
cargo run -p gemstone-rs-cli -- profile show default examples/codegen/gemstone-
rs.codegen-profiles.json
cargo run -p gemstone-rs-cli -- profile resolve default examples/codegen/gemstone-
rs.codegen-profiles.json
cargo run -p gemstone-rs-cli -- profile check examples/codegen/gemstone-rs.codegen-
profiles.json
cargo run -p gemstone-rs-cli -- profile check --json examples/codegen/gemstone-
rs.codegen-profiles.json
cargo run -p gemstone-rs-cli -- codegen check-profile default examples/codegen/
gemstone-rs.codegen-profiles.json
```

```
cargo run -p gemstone-rs-cli -- codegen explain-profile --json default examples/
codegen/gemstone-rs.codegen-profiles.json
```

For an installed CLI:

```
gemstone-rs profile sample
gemstone-rs profile init gemstone-rs.codegen-profiles.json
gemstone-rs profile validate gemstone-rs.codegen-profiles.json
gemstone-rs profile validate --json gemstone-rs.codegen-profiles.json
gemstone-rs profile list gemstone-rs.codegen-profiles.json
gemstone-rs profile show default gemstone-rs.codegen-profiles.json
gemstone-rs profile resolve default gemstone-rs.codegen-profiles.json
gemstone-rs profile check gemstone-rs.codegen-profiles.json
gemstone-rs profile check --json gemstone-rs.codegen-profiles.json
gemstone-rs codegen check-profile default gemstone-rs.codegen-profiles.json
gemstone-rs codegen explain-profile --json default gemstone-rs.codegen-profiles.json
```

Expected output:

```
profile ok: examples/codegen/gemstone-rs.codegen-profiles.json (3 profiles: default,
object-wrapper, bridge-mapping)
profile check: examples/codegen/gemstone-rs.codegen-profiles.json (3 profiles)
summary: 3 ok, 0 stale, 0 errors, 3 total
ok      default config=examples/codegen/gemstone-rs.codegen     output=examples/
codegen/generated/gemstone_wrappers.rs
```

The JSON form includes aggregate fields so CI and editor integrations can fail with a concise summary:

```
{"success":true,"ok":true,"profileCount":3,"okCount":3,"staleCount":0,"errorCount":0}
```

The profile check JSON shape is shared by the CLI, explorer, and VS Code Workbench. Its schema lives at:

```
schemas/gemstone-rs.profile-check.schema.json
```

Shape

```
{
  "kind": "gemstone-rs-explorer-codegen-profiles",
  "version": 1,
  "profiles": [
    {
      "name": "default",
      "config": "examples/codegen/gemstone-rs.codegen",
```

```
      "root": "",
      "mapped": "BookingDraft",
      "className": "Object"
    }
  ]
}
```

Top-level fields:

- `kind` must be `gemstone-rs-explorer-codegen-profiles`.
- `version` must be `1`.
- `profiles` must be an array.

Profile fields:

- `name` is required, string-valued, non-empty, and unique.
- `config` is optional and string-valued.
- `root` is optional and string-valued.
- `mapped` is optional and string-valued.
- `className` is optional and string-valued.

Unknown top-level or profile fields are rejected. Non-string profile fields are rejected.

VS Code

Use `GemStone RS: Show Sample Project Profiles` to open the built-in sample in an untitled JSON editor, `GemStone RS: Create Project Profiles` to write the sample to a project file, and `GemStone RS: Validate Project Profiles` to run the same CLI validator and show the result in the GemStone RS output panel. `GemStone RS: List Project Profiles`, `GemStone RS: Show Project Profile`, and `GemStone RS: Resolve Project Profile` render parsed profile details through the same CLI parser, so you can check the active config/root/mapped/class fields and resolved config path without opening the explorer. Profile-specific commands use a QuickPick populated from the profile file. The workbench also contributes JSON validation for files named `gemstone-rs.codegen-profiles.json`, using the packaged schema copy in `vscode-gemstone-rs-workbench/schemas/`.

Without the extension, associate the schema with the profile file in VS Code:

```
{
  "json.schemas": [
    {
      "fileMatch": ["gemstone-rs.codegen-profiles.json"],
      "url": "./schemas/gemstone-rs.codegen-profiles.schema.json"
    }
  ]
}
```

Explorer Safety

The explorer validates project profile JSON before saving. It also rejects write paths containing `..` after URL decoding. Relative profile writes stay inside the configured `--codegen-root`; absolute write targets require `--allow-absolute-write-paths`.

Explorer Guide

`gemstone-rs-explorer` is a local-only HTTP explorer built on the Rust API. It is useful for proving the browser, inspect, eval, and codegen surfaces before embedding richer tooling in VS Code or another frontend.

Install and Run

```
cargo install gemstone-rs-explorer
gemstone-rs-explorer --port 8787
gemstone-rs-explorer --env-file .env.gemstone-rs --port 8787
```

From a checkout:

```
cargo run -p gemstone-rs-explorer -- --port 8787
```

If you launch the explorer outside the project checkout, point codegen discovery and relative config paths at the checkout explicitly:

```
gemstone-rs-explorer --port 8787 --codegen-root /path/to/gemstone-rs
```

Open:

```
http://127.0.0.1:8787/
```

The home page is a small browser UI over the same JSON endpoints. It can:

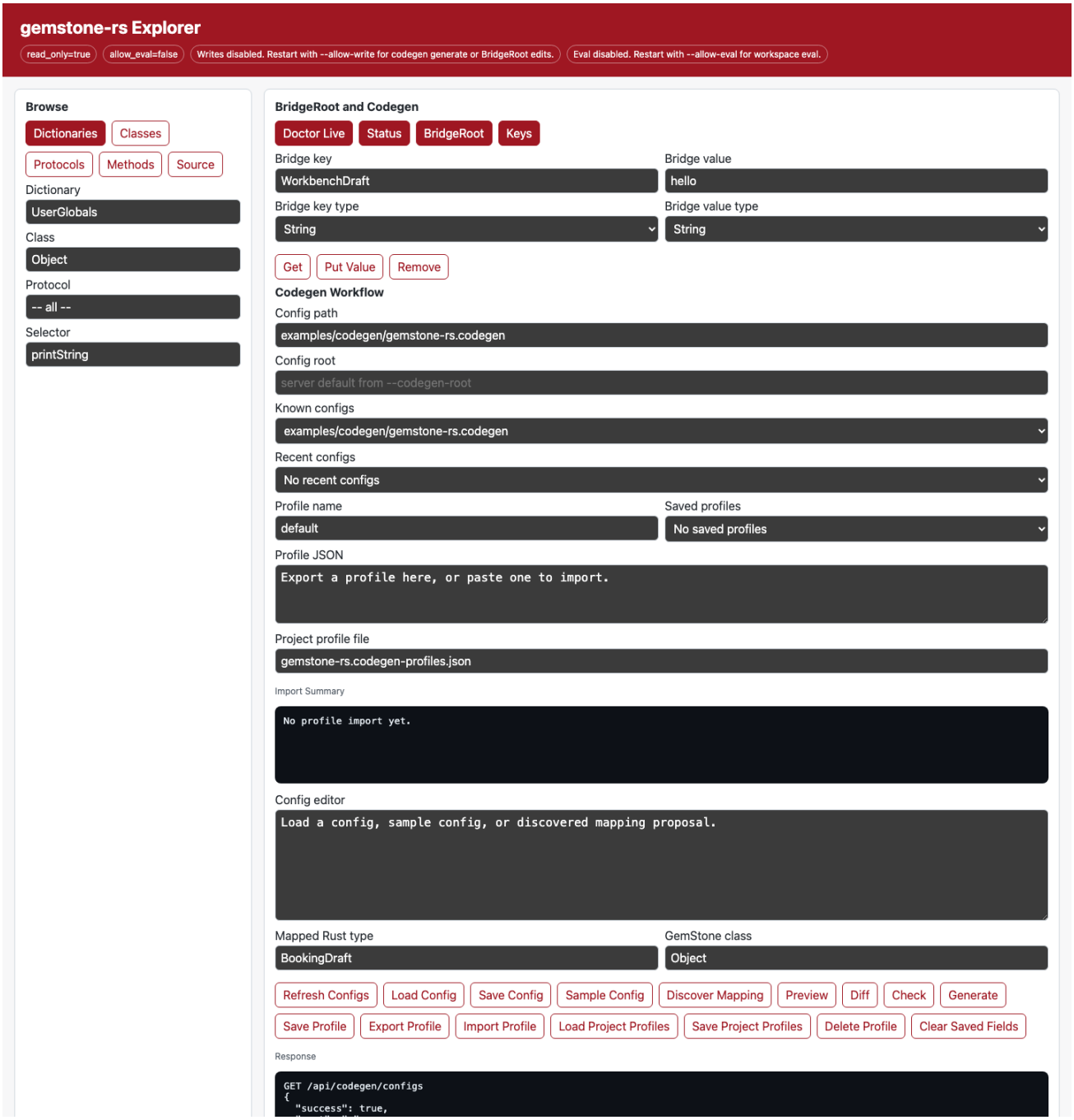
- browse dictionaries, classes, protocols, methods, and source
- run a setup assistant that checks the env file, live configuration, GCI

  library, codegen config, and project profiles

- run doctor/status checks
- inspect BridgeRoot and list keys

- run codegen sample/discover/preview/diff/check/generate from an editable config path
- refresh a picker of known `.codegen` files and load one without typing the path by hand
- set a project codegen root through `--codegen-root`, or override it from the page before refreshing configs
- reuse recently loaded, saved, previewed, diffed, checked, or generated config paths from local browser storage
- save named codegen profiles containing config root, config path, mapped Rust type, and GemStone class
- export and import codegen profiles as versioned JSON for sharing between browsers or teammates
- load project profile files from the codegen root, and save them back when the explorer runs with `--allow-write`
- load and save the selected codegen config file; saving is still gated by `--allow-write`, accepts a POST body for larger configs, and validates the config before writing
- render generated wrapper source, generated mapping config, colored unified diff output, and a side-by-side diff in a dedicated detail pane
- remember the current browser fields locally across reloads
- put/remove BridgeRoot values with explicit string/symbol key policy and string/small-int/bool value policy when `--allow-write` is enabled

Screenshot:



Refresh it with:

```
make screenshots
```

See Screenshot Workflow for the repeatable capture process.

Safety Defaults

The explorer:

- binds to `127.0.0.1` by default
- rejects non-loopback hosts
- starts read-only
- requires `--allow-eval` before workspace evaluation
- requires `--allow-write` before codegen writes
- supports an optional local auth token with `--auth-token` or `--auth-token-env`
- reads GemStone credentials from the same `GS_*` environment as the CLI

Do not expose it publicly. The token option is for local defense in depth on a loopback machine; it is not a substitute for TLS, user accounts, or a production auth layer.

To require a token for all explorer pages and APIs except `/health`:

```
export GEMSTONE_RS_EXPLORER_TOKEN='replace-with-a-local-random-token'
gemstone-rs-explorer --auth-token-env GEMSTONE_RS_EXPLORER_TOKEN
```

Then use either a query token for browser workflows:

```
open 'http://127.0.0.1:8787/?token=replace-with-a-local-random-token'
curl -s 'http://127.0.0.1:8787/api/config?token=replace-with-a-local-random-token'
```

Or a header for scripts:

```
curl -s \
  -H 'X-GemStone-RS-Token: replace-with-a-local-random-token' \
  http://127.0.0.1:8787/api/config
curl -s \
  -H 'Authorization: Bearer replace-with-a-local-random-token' \
  http://127.0.0.1:8787/api/status
```

Browse Endpoints

Run these from a second terminal:

```
curl -s http://127.0.0.1:8787/health
curl -s http://127.0.0.1:8787/api/config
curl -s 'http://127.0.0.1:8787/api/setup/assistant?config=examples/codegen/gemstone-
rs.codegen&profile_file=examples/codegen/gemstone-rs.codegen-profiles.json'
```

```
curl -s http://127.0.0.1:8787/api/doctor
curl -s 'http://127.0.0.1:8787/api/doctor?live=1'
curl -s http://127.0.0.1:8787/api/status
curl -s http://127.0.0.1:8787/api/browse/dictionaries
curl -s 'http://127.0.0.1:8787/api/browse/classes?dictionary=UserGlobals'
curl -s 'http://127.0.0.1:8787/api/browse/protocols?class=Object&meta=0'
curl -s 'http://127.0.0.1:8787/api/browse/methods?class=Object&protocol=--%20all%20--
&meta=0'
curl -s 'http://127.0.0.1:8787/api/browse/source?
class=Object&selector=printString&meta=0'
curl -s 'http://127.0.0.1:8787/api/inspect?oop=20'
```

Expected shapes:

```
{"status":"ok"}
{"host":"127.0.0.1","port":8787,"readOnly":true,"allowEval":false}
{"success":true,"environment":{"GS_PASSWORD":{"status":"set","masked":true}}}
{"connected":true,"sessionId":12345,"needsCommit":false,"inTransaction":false}
{"success":true,"dictionaries":["UserGlobals"]}
```

Eval

Eval is disabled by default:

```
gemstone-rs-explorer --allow-eval
```

Then:

```
curl -s 'http://127.0.0.1:8787/api/eval?source=3%20%2B%204'
```

Expected:

```
{"type":"smallInt","value":7}
```

Codegen Endpoints

The setup actions on the home page can show the same safe `GS_*` environment template as `gemstone-rs env sample`, or write `.env.gemstone-rs` when the explorer was started with `--allow-write`. Start the explorer with `--env-file .env.gemstone-rs` to use that file for all live browse, inspect, BridgeRoot, and codegen discovery actions. `Run Setup Assistant` checks the env file, GemStone configuration, GCI library, selected codegen config, and project profile file in one response.

The home page includes a Codegen Workflow panel. Set `Config path`, optionally set `Config root`, or start the server with `--codegen-root`. `Refresh Configs` and the `Known configs` picker choose an existing `.codegen` file relative to that root. The `Recent configs` picker remembers paths used by Load, Save, Preview, Diff, Check, and Generate in local browser storage. `Saved profiles` store the root, config path, mapped Rust type, and GemStone class as a named local browser profile. `Export Profile` writes a versioned JSON payload to `Profile JSON`; paste that JSON into another explorer and use `Import Profile` to merge it into saved profiles. `Load Project Profiles` reads `gemstone-rs.codegen-profiles.json` from the codegen root by default, while `Save Project Profiles` writes the current saved profiles back to that file when the explorer runs with `--allow-write`. Optionally enter a mapped Rust type and GemStone class, then use:

- `Config root` to override the server default for config discovery and

  relative config paths

- `Refresh Configs` to scan the project root for known `.codegen` files
- `Recent configs` to return to the last few config paths used in this browser
- `Save Profile` and `Saved profiles` to switch between named codegen workflows
- `Export Profile` and `Import Profile` to share a codegen workflow as JSON
- `Load Project Profiles` and `Save Project Profiles` to use a committed

  project-level profile file; the server rejects invalid profile schemas and the browser reports which profiles are new, replaced, or unchanged

- `Check Project Profiles` to verify every profile in the project profile file

  and show ok/stale/error counts in one report; the detail pane renders a table with profile name, config path, output file, freshness, and per-profile Preview, Diff, Check, and Generate actions

- `Load Config` to read the selected config file into the editor
- `Save Config` to POST the editor contents, validate them, and write the file

  when the explorer was started with `--allow-write`

- `Sample Config` to view the starter config format
- `Discover Mapping` to ask a live stone for a mapping config proposal
- `Explain` to render a structured codegen summary, including output path,

  generated test stubs, wrappers, return helpers, and mapped fields

- `Explain Profile` to render the same structured summary after resolving a

  named profile from the project profile file

- `Preview` to inspect generated Rust wrappers without writing files
- `Preview Profile` to inspect generated wrappers after resolving a named

  project profile

- `Diff` to compare generated output with the committed file; the detail pane shows both the exact unified diff and a side-by-side view for review
- `Diff Profile` to compare generated output from a named project profile
- `Check` to fail when generated output is stale
- `Check Profile` to run the stale-output check from a named project profile
- `Generate` to write wrappers when the explorer was started with `--allow-write`
- `Generate Profile` to resolve a named project profile and write its wrappers when the explorer was started with `--allow-write`

Read-only endpoints:

```
curl -s http://127.0.0.1:8787/api/codegen/sample
curl -s 'http://127.0.0.1:8787/api/codegen/configs?root=.'
curl -s 'http://127.0.0.1:8787/api/codegen/profiles?profile_file=gemstone-rs.codegen-
profiles.json'
curl -s 'http://127.0.0.1:8787/api/codegen/profiles/check?profile_file=examples/
codegen/gemstone-rs.codegen-profiles.json'
curl -s 'http://127.0.0.1:8787/api/codegen/config?config=examples/codegen/gemstone-
rs.codegen'
curl -s 'http://127.0.0.1:8787/api/codegen/explain?config=examples/codegen/gemstone-
rs.codegen'
curl -s 'http://127.0.0.1:8787/api/codegen/explain-profile?
profile=default&profile_file=examples/codegen/gemstone-rs.codegen-profiles.json'
curl -s 'http://127.0.0.1:8787/api/codegen/preview?config=examples/codegen/gemstone-
rs.codegen'
curl -s 'http://127.0.0.1:8787/api/codegen/preview-profile?
profile=default&profile_file=examples/codegen/gemstone-rs.codegen-profiles.json'
curl -s 'http://127.0.0.1:8787/api/codegen/diff?config=examples/codegen/gemstone-
rs.codegen'
curl -s 'http://127.0.0.1:8787/api/codegen/diff-profile?
profile=default&profile_file=examples/codegen/gemstone-rs.codegen-profiles.json'
curl -s 'http://127.0.0.1:8787/api/codegen/check?config=examples/codegen/gemstone-
rs.codegen'
curl -s 'http://127.0.0.1:8787/api/codegen/check-profile?
profile=default&profile_file=examples/codegen/gemstone-rs.codegen-profiles.json'
curl -s 'http://127.0.0.1:8787/api/codegen/discover-mapping?
mapped=BookingDraft&class=Object'
```

The repository includes a sample project profile file:

```
examples/codegen/gemstone-rs.codegen-profiles.json
```

Expected read-only results include the config list and a current generated output check:

```
{"success":true,"root":".","configs":["examples/codegen/gemstone-rs.codegen"]}
{"success":true,"exists":true,"upToDate":true}
{"success":true,"ok":true,"profileCount":3,"okCount":3,"staleCount":0,"errorCount":0}
```

Write-gated endpoint:

```
gemstone-rs-explorer --allow-write
```

```
curl -s 'http://127.0.0.1:8787/api/codegen/generate?config=examples/codegen/gemstone-
rs.codegen'
curl -s 'http://127.0.0.1:8787/api/codegen/generate-profile?
profile=default&profile_file=examples/codegen/gemstone-rs.codegen-profiles.json'
curl -s -X POST \
  'http://127.0.0.1:8787/api/env/write?path=.env.gemstone-rs'
curl -s -X POST \
  --data-binary @gemstone-rs.codegen-profiles.json \
  'http://127.0.0.1:8787/api/codegen/profiles/save?profile_file=gemstone-rs.codegen-
profiles.json'
curl -s -X POST \
  --data-binary @examples/codegen/gemstone-rs.codegen \
  'http://127.0.0.1:8787/api/codegen/config/save?config=examples/codegen/
draft.codegen'
```

Expected:

```
{"success":true,"output":"examples/codegen/generated/
gemstone_wrappers.rs","bytes":1234}
{"success":true,"config":"examples/codegen/draft.codegen","bytes":512}
```

Project profile files use this shape:

```
{"kind":"gemstone-rs-explorer-codegen-profiles","version":1,"profiles":
[{"name":"default","config":"examples/codegen/gemstone-
rs.codegen","root":"","mapped":"BookingDraft","className":"Object"}]}
```

Validate the file before sharing or saving it from CI:

```
gemstone-rs profile sample
gemstone-rs profile init gemstone-rs.codegen-profiles.json
gemstone-rs profile validate gemstone-rs.codegen-profiles.json
gemstone-rs profile validate --json gemstone-rs.codegen-profiles.json
gemstone-rs profile list gemstone-rs.codegen-profiles.json
gemstone-rs profile show default gemstone-rs.codegen-profiles.json
gemstone-rs profile resolve default gemstone-rs.codegen-profiles.json
gemstone-rs profile check gemstone-rs.codegen-profiles.json
```

```
gemstone-rs profile check --json gemstone-rs.codegen-profiles.json
gemstone-rs codegen check-profile default gemstone-rs.codegen-profiles.json
```

The schema reference is Codegen Profile Schema.

For write endpoints, relative `config=` and `profile_file=` paths are resolved inside the codegen root. Paths containing `..` are rejected after URL decoding, `root=...` traversal is rejected as well, and absolute write paths require starting the explorer with `--allow-absolute-write-paths`.

The browser UI also asks for confirmation before BridgeRoot writes, env-file writes, profile saves, config saves, and codegen generate actions. Server-side guards still decide the real access policy: write endpoints return `403` unless the explorer was started with `--allow-write`.

Project profile saves validate the JSON before writing. The top-level object must contain only `kind`, `version`, and `profiles`; profile names are required and unique; and each profile may contain only string-valued `name`, `config`, `root`, `mapped`, and `className` fields.

BridgeRoot Endpoints

BridgeRoot inspection and mapping-config preview use the same live session configuration as the browser endpoints:

```
curl -s http://127.0.0.1:8787/api/bridge/root
curl -s http://127.0.0.1:8787/api/bridge/keys
curl -s 'http://127.0.0.1:8787/api/bridge/get?key=BookingDraft'
curl -s 'http://127.0.0.1:8787/api/bridge/get?key=BookingDraft&key_type=Symbol'
curl -s 'http://127.0.0.1:8787/api/bridge/mapping-config?mapped=BookingDraft'
```

The browser UI exposes `Bridge key type` and `Bridge value type` controls so you can explicitly test string-key, symbol-key, string-value, symbol-value, small-int-value, and bool-value mappings before encoding them in a reusable config file. It persists the selected key, value, class, selector, config, and mapping fields in browser local storage; use `Clear Saved Fields` to reset the page back to the documented defaults.

Writes are disabled unless the explorer is started with `--allow-write`:

```
gemstone-rs-explorer --allow-write
curl -s 'http://127.0.0.1:8787/api/bridge/put?key=WorkbenchDraft&value=hello'
curl -s 'http://127.0.0.1:8787/api/bridge/put?
key=WorkbenchState&value=ready&value_type=Symbol'
curl -s 'http://127.0.0.1:8787/api/bridge/put?
key=WorkbenchCount&value=7&value_type=SmallInt'
curl -s 'http://127.0.0.1:8787/api/bridge/put?
key=WorkbenchApproved&value=true&value_type=Bool'
curl -s 'http://127.0.0.1:8787/api/bridge/remove?key=WorkbenchDraft'
```

Expected shapes:

```
{"success":true,"name":"GemStoneRsBridgeRoot","oop":1234,"identityId":1}
{"success":true,"root":"GemStoneRsBridgeRoot","keys":
[{"printString":"BookingDraft"}]}
{"oop":5678,"classOop":9012,"printString":"aDictionary(...)"}
{"success":true,"config":"mapped = BookingDraft ..."}
{"success":true,"root":"GemStoneRsBridgeRoot","key":"WorkbenchDraft","keyType":"Strin
g","valueType":"String","oop":3456}
```

Product Direction

The explorer should remain a separate binary crate. The core `gemstone-rs` library stays focused on safe GemStone API access, while the explorer proves higher-level browsing and codegen workflows.

Good next steps:

- short GIFs showing profile import, codegen preview, and BridgeRoot checks
- deeper VS Code webview integration

VS Code Workbench

`gemstone-rs Workbench` is the VS Code companion for the Rust CLI and local explorer. It keeps the CLI as the stable contract and adds a sidebar, output panel, generated-file previews, and codegen diff prompts.

Marketplace:

```
https://marketplace.visualstudio.com/items?itemName=unicompute.gemstone-rs-workbench
```

Setup

For installed CLI tools:

```
{
  "gemstoneRs.cliPath": "gemstone-rs",
  "gemstoneRs.explorerPath": "gemstone-rs-explorer",
  "gemstoneRs.codegenConfig": "gemstone-rs.codegen",
```

```
  "gemstoneRs.bridgeRoot": "GemStoneRsBridgeRoot"
}
```

For a source checkout:

```
{
  "gemstoneRs.checkoutPath": "/path/to/gemstone-rs",
  "gemstoneRs.useCargo": true,
  "gemstoneRs.codegenConfig": "examples/codegen/gemstone-rs.codegen",
  "gemstoneRs.bridgeRoot": "GemStoneRsBridgeRoot"
}
```

Use the same `GS_*` environment as the CLI.

Sidebar Tree

Open the GemStone RS activity bar item.

The sidebar contains:

- `Dictionaries`
- `Codegen Config`
- `Explorer`

`Dictionaries` expands live dictionaries, classes, protocols, and methods. Selecting a method opens its source in an untitled Smalltalk editor.

`Codegen Config` exposes:

- Discover from Live Stone
- BridgeRoot name from `gemstoneRs.bridgeRoot`
- Generate Mapping Config
- Preview BridgeRoot
- List BridgeRoot Keys
- Put BridgeRoot String
- Put BridgeRoot Symbol
- Put BridgeRoot SmallInt
- Put BridgeRoot Bool
- Remove BridgeRoot Key
- Run Generated Mapping Example
- Preview Wrappers
- Diff Generated Output
- Check Freshness
- Generate Wrappers
- Load Project Profiles
- Save Project Profiles

- Export Codegen Profile
- Show Sample Project Profiles
- Create Project Profiles
- Validate Project Profiles
- List Project Profiles
- Show Project Profile
- Resolve Project Profile
- Check Project Profiles
- Open Codegen Docs

`Explorer` exposes:

- Doctor
- Verify Setup
- Verify Live Setup
- Eval Smalltalk
- Generate Explorer Auth Token
- Clear Explorer Auth Token
- Launch Explorer
- Open Explorer Webview

Command Palette

Commands:

- `GemStone RS: Verify Setup`
- `GemStone RS: Verify Live Setup`
- `GemStone RS: Verify Strict Setup`
- `GemStone RS: Doctor`
- `GemStone RS: Show Environment Template`
- `GemStone RS: Copy Environment Template`
- `GemStone RS: Write .env.gemstone-rs`
- `GemStone RS: Eval Smalltalk`
- `GemStone RS: Browse Dictionaries`
- `GemStone RS: Browse Classes`
- `GemStone RS: Codegen Init`
- `GemStone RS: Codegen Discover`
- `GemStone RS: Generate Mapping Config`
- `GemStone RS: Preview BridgeRoot`
- `GemStone RS: List BridgeRoot Keys`
- `GemStone RS: Put BridgeRoot String`
- `GemStone RS: Put BridgeRoot Symbol`
- `GemStone RS: Put BridgeRoot SmallInt`
- `GemStone RS: Put BridgeRoot Bool`
- `GemStone RS: Remove BridgeRoot Key`

- `GemStone RS: Run Generated Mapping Example`
- `GemStone RS: Codegen Preview`
- `GemStone RS: Codegen Diff`
- `GemStone RS: Codegen Check`
- `GemStone RS: Codegen Explain`
- `GemStone RS: Codegen Generate`
- `GemStone RS: Codegen Preview Profile`
- `GemStone RS: Codegen Diff Profile`
- `GemStone RS: Codegen Check Profile`
- `GemStone RS: Codegen Explain Profile`
- `GemStone RS: Codegen Generate Profile`
- `GemStone RS: Load Project Profiles`
- `GemStone RS: Save Project Profiles`
- `GemStone RS: Export Codegen Profile`
- `GemStone RS: Show Sample Project Profiles`
- `GemStone RS: Create Project Profiles`
- `GemStone RS: Validate Project Profiles`
- `GemStone RS: List Project Profiles`
- `GemStone RS: Show Project Profile`
- `GemStone RS: Resolve Project Profile`
- `GemStone RS: Check Project Profiles`
- `GemStone RS: Generate Explorer Auth Token`
- `GemStone RS: Clear Explorer Auth Token`
- `GemStone RS: Launch Explorer`
- `GemStone RS: Open Explorer Webview`
- `GemStone RS: Open Method Source`
- `GemStone RS: Open Codegen Docs`

`GemStone RS: Verify Setup` and `GemStone RS: Doctor` both run the CLI `gemstone-rs doctor` report, so the workbench setup view and terminal setup use the same diagnostic path. `GemStone RS: Verify Live Setup` runs `gemstone-rs doctor --live` when credentials and a reachable stone are available. `GemStone RS: Verify Strict Setup` runs `gemstone-rs doctor --strict` for CI-style validation of explicit stone and GCI library settings. All setup checks add the active checkout, CLI, explorer, codegen config, and profile paths, plus whether the configured `gemstoneRs.envFile` exists. When that file exists, Workbench passes `--env-file` automatically to CLI commands and explorer startup. The report also includes the configured BridgeRoot name used by BridgeRoot commands and explorer probes. The checks offer result actions to copy the full report, copy a safe `GS_*` environment export script with a placeholder password, or open the extension settings.

`GemStone RS: Run Setup Assistant` calls the running local explorer at `/api/setup/assistant` and renders the structured env-file, GemStone configuration, GCI library, codegen config, and project profile checks in the output panel. Start the explorer first with `GemStone RS: Launch Explorer`.

`GemStone RS: Show Environment Template`, `GemStone RS: Copy Environment Template`, and `GemStone RS: Write .env.gemstone-rs` call the CLI `gemstone-rs env sample/write` commands. The template uses current non-secret values when available and keeps passwords as placeholders.

Codegen Workflow

1. Set `gemstoneRs.codegenConfig`.
2. Run `GemStone RS: Codegen Discover` or edit the config manually.
3. Run `GemStone RS: Codegen Preview`.
4. Run `GemStone RS: Codegen Diff`.
5. Run `GemStone RS: Codegen Explain`.
6. Run `GemStone RS: Codegen Generate`.

`Codegen Generate` runs the diff first. If output would change, it opens the diff and asks before writing.

`Codegen Explain` runs `codegen explain --json` and renders the output path, generated test stubs, wrapper classes, selectors, return types, and BridgeRoot mappings in the output panel. Result actions can copy the summary, copy the raw JSON, open the JSON in an editor, or open the config file.

When a project profile file is checked in, use the profile variants instead: `Codegen Preview Profile`, `Codegen Diff Profile`, `Codegen Check Profile`, `Codegen Explain Profile`, and `Codegen Generate Profile`. They prompt for a profile name and `gemstone-rs.codegen-profiles.json`, resolve the profile's config/root fields, and then run the same codegen operation.

Object Mapping Workflow

Use `GemStone RS: Generate Mapping Config` when you want the live stone to inspect a GemStone class and propose a `BridgeMapped` config. The command asks for:

- config path
- Rust struct name
- GemStone class reference, such as `Object` or `UserGlobals:OkzBooking`

Use `GemStone RS: Preview BridgeRoot` after starting the local explorer. It opens:

```
http://127.0.0.1:8787/api/bridge/root?root=GemStoneRsBridgeRoot
```

Use `GemStone RS: List BridgeRoot Keys` for the same inspection path without opening a browser; it runs `gemstone-rs bridge keys --root <bridge-root>` and writes the key OOPs, class OOPs, `printString` values, and identity ids to the output panel.

Set `gemstoneRs.bridgeRoot` when a project uses a custom bridge dictionary global. The workbench passes that value to CLI commands as `--root` and to the explorer API as the `root=` query parameter.

Use `GemStone RS: Put BridgeRoot String` when you want a quick live-write smoke test from VS Code. It asks for a key, whether that key is a String or Symbol, and a string value, then runs:

```
gemstone-rs bridge put-string <key> <value> --key-type String --root
GemStoneRsBridgeRoot
```

Use `GemStone RS: Put BridgeRoot Symbol` for the same workflow when the stored value should be a GemStone Symbol:

```
gemstone-rs bridge put-symbol <key> <value> --key-type String --root
GemStoneRsBridgeRoot
```

Use `GemStone RS: Put BridgeRoot SmallInt` or `GemStone RS: Put BridgeRoot Bool` when the stored value should be a GemStone SmallInteger or Boolean:

```
gemstone-rs bridge put-smallint <key> <value> --key-type String --root
GemStoneRsBridgeRoot
gemstone-rs bridge put-bool <key> <value> --key-type String --root
GemStoneRsBridgeRoot
```

Use `GemStone RS: Remove BridgeRoot Key` to remove one BridgeRoot key after a String/Symbol key-type prompt and confirmation prompt:

```
gemstone-rs bridge remove <key> --key-type String --root GemStoneRsBridgeRoot
```

Use `GemStone RS: Run Generated Mapping Example` to run:

```
cargo run -p gemstone-rs --example generated_mapping_app
```

Embedded Explorer Webview

Use `GemStone RS: Launch Explorer` first. It starts `gemstone-rs-explorer` in a terminal and opens the browser. Then use `GemStone RS: Open Explorer Webview` to embed the local explorer inside VS Code.

The webview points at:

```
http://127.0.0.1:8787/
```

If you set `gemstoneRs.explorerAuthToken`, `GemStone RS: Launch Explorer` starts the server with `--auth-token-env GEMSTONE_RS_EXPLORER_TOKEN` so the token is not printed in the terminal command. Browser and webview URLs include the matching `token=` query parameter. Use `GemStone RS: Generate Explorer Auth Token` to create a local random token, save it to VS Code settings, and copy it to the clipboard. Use `GemStone RS: Clear Explorer Auth Token` to return to the default loopback-only, no-token mode.

It is still the same loopback-only explorer process. If the explorer is not running, the webview will show a connection error and the output panel will show the URL to start.

The embedded page now acts as the main IDE surface for the local explorer. The iframe remains the full browser UI, while the side inspector can query explorer status, run setup checks, inspect project profile freshness, preview Codegen JSON, inspect diffs, check generated freshness, and inspect BridgeRoot keys for the configured root. The same shell can hand off to native VS Code commands for generated wrapper preview, diff, check, generate-with-confirmation, profile checks, docs, and opening the last generated output file reported by the explorer.

The explorer page can load and save the selected codegen config file, posting the editor contents as the request body; saves still require the explorer to run with `--allow-write`. Use `Refresh Configs` in the embedded page to populate the `Known configs` picker from the explorer process working tree, or set `Config root` in the page when the explorer was launched from a different directory. The page also keeps a local `Recent configs` picker for paths used by Load, Save, Preview, Diff, Check, and Generate. Use `Save Profile` when you want to switch between named codegen workflows; a profile stores the config root, config path, mapped Rust type, and GemStone class in local browser storage. `Export Profile` writes a JSON payload into `Profile JSON`, and `Import Profile` merges pasted profile JSON from another browser or teammate. `Load Project Profiles` reads `gemstone-rs.codegen-profiles.json` from the configured codegen root, and `Save Project Profiles` writes the saved profile list back when the explorer was launched with `--allow-write`. Project profile payloads are schema-validated before writing, and imports report which profiles are new, replaced, or unchanged. It remembers its current fields locally, so repeated codegen or BridgeRoot checks keep the same config path and class selection.

Use `GemStone RS: Show Sample Project Profiles` to open the built-in sample from `gemstone-rs profile sample`, `GemStone RS: Create Project Profiles` to write that sample into a project file with `gemstone-rs profile init`, and `GemStone RS: Validate Project Profiles` when you want a direct CLI check without opening the explorer. Validation runs `gemstone-rs profile validate` against the selected profile file and writes the result to the GemStone RS output panel. `GemStone RS: List Project Profiles` and `GemStone RS: Show Project Profile` run `gemstone-rs profile list/show` and render the parsed config/root/mapped/class fields in the same output panel. `GemStone RS: Resolve Project Profile` runs `gemstone-rs profile resolve` and shows the config path that profile-driven codegen will use. `GemStone RS: Check Project Profiles` runs `gemstone-rs profile check --json` across every profile and renders an aggregate summary with ok, stale, and error counts before you commit.

The result message can copy the report or open the profile file for quick repair. Profile-specific commands use a QuickPick populated from the project profile file. The extension also contributes JSON validation for files named `gemstone-rs.codegen-profiles.json` and profile check reports named `gemstone-rs.profile-check.json`.

Develop the Extension

```
cd vscode-gemstone-rs-workbench
npm ci
npm run check
npm run test:smoke
npm run package -- --out gemstone-rs-workbench-0.3.4.vsix
```

From the repository root:

```
make vscode-package
```

Later Feature Work

The first embedded webview is available. Next work should make the webview more IDE-like: generate wrapper diffs before writing, edit codegen selections, inspect BridgeRoot payloads visually, and keep browser fallback behavior for users who prefer the external explorer.

Screenshot Workflow

The repository keeps screenshots in `docs/assets/` so the README, guides, PDFs, Marketplace text, and articles can point at stable image paths.

Explorer Screenshot

`docs/assets/explorer-home.png` is captured from the local explorer at a 1440x1500 viewport.

Install Playwright once:

```
python3 -m pip install playwright
python3 -m playwright install chromium
```

If Playwright is not installed, the capture script falls back to a local Chrome, Chromium, or Microsoft Edge executable when one is available.

Then refresh the screenshot:

```
make screenshots
```

The target starts:

```
cargo run -p gemstone-rs-explorer -- --port 8787
```

It waits for:

```
http://127.0.0.1:8787/health
```

Then it writes:

```
docs/assets/explorer-home.png
```

If the explorer is already running, capture it without starting another server:

```
python3 scripts/capture_explorer_screenshots.py --no-server --url http://
127.0.0.1:8787/
```

Run a dry path check without launching a browser:

```
python3 scripts/capture_explorer_screenshots.py --check
```

The screenshot path does not require GemStone credentials because the explorer home page renders before live endpoints are called. Live browse/codegen data still requires the normal `GS_*` environment.

After Capture

Regenerate PDFs after refreshing screenshots:

```
python3 docs/build_pdf_docs.py
```

For a release pass, run:

```
make verify
make vscode-package
```

Performance and Safety

`gemstone-rs` keeps the low-level GemStone/S GCI boundary explicit. The goal is to make Rust services and tooling fast without hiding session, transaction, or threading rules.

Dynamic GCI Loading

The GCI library is loaded at runtime. Configure it with one of:

```
export GS_LIB=/path/to/GemStone64Bit/lib
export GS_LIB_PATH=/path/to/GemStone64Bit/lib
export GEMSTONE=/path/to/GemStone64Bit
```

The loader already reports which source selected `libgcirpc` in `gemstone-rs doctor`: explicit config, `GS_LIB_PATH`, `GS_LIB`, or `GEMSTONE/lib`, plus the exact path or directory searched. It should keep improving in these directions:

- detect architecture mismatches early
- avoid requiring GemStone headers at Rust build time
- keep all unsafe C ABI calls isolated in `gemstone-gci`

Session Threading

`Session` is deliberately non-`Send` and non-`Sync`. Treat a session as owned by the thread that logged it in.

For web servers:

- use `spawn_blocking` for synchronous GemStone calls from async Rust
- prefer one session per request until a proven session pool exists
- keep transaction boundaries explicit
- do not share a live session between async tasks

Benchmark Smoke Test

Use the lightweight benchmark smoke script for a coarse local sanity check:

```
ITERATIONS=20 scripts/benchmark_smoke.sh "3 + 4"
```

This measures CLI startup, login, eval, and logout together. It is not a microbenchmark. A future benchmark suite should separate:

- GCI library load time
- login/logout time
- eval/perform latency
- string and array marshalling
- BridgeRoot read/write latency
- codegen wrapper overhead

Rust vs Python

Compare `gemstone-rs` and `gemstone-py` only with equivalent workloads:

- same stone
- same host
- same GemStone user
- same expression or selector
- warm and cold runs reported separately

The first useful comparison is:

```
Rust:   cargo run -p gemstone-rs-cli -- eval "3 + 4"
Python: python -m gemstone_py.cli eval "3 + 4"
```

After that, compare generated-wrapper calls and BridgeRoot mapping operations.

Safety Checklist

- Keep `Session` non-`Send` and non-`Sync`.
- Keep `unsafe` in `gemstone-gci`.
- Prefer typed `Value` / `Oop` conversions over raw integer handling.
- Commit or abort explicitly after writes.
- Bind local tools to `127.0.0.1` by default.
- Keep explorer eval/write operations opt-in.

Shared Core Integration

The long-term direction is:

```
gemstone-gci
  Low-level dynamic libgcirpc loader and raw GCI calls.

gemstone-rs
  Safe Rust API: Config, Session, Oop, Value, browser, codegen, BridgeRoot.

gemstone-py-native
  Thin PyO3 wrapper around the Rust core.

gemstone-py
  Python API, async/web adapters, examples, docs, and compatibility layer.
```

This keeps Python and Rust from growing separate native bridges.

What Should Move Into Rust

- dynamic GCI library loading
- OOP constants and conversions
- session login/logout
- eval, execute, perform
- global get/put
- string, symbol, array, and dictionary marshalling
- transaction commit/abort
- browser primitives
- codegen discovery primitives

What Should Stay Python-Specific

- Pythonic `Session` and async wrappers
- FastAPI, Litestar, Django examples
- Python package extras such as `gemstone-py[fast]`
- backward-compatible return behavior
- Python docs and notebooks

PyO3 Wrapper Shape

`gemstone-py-native` should become a small adapter:

```
Python call -> PyO3 wrapper -> gemstone-rs Session -> gemstone-gci -> libgcirpc
```

The wrapper should expose stable operations first:

- `login(config)`
- `eval(source)`
- `execute(source)`
- `perform(oop, selector, args)`
- `commit()`
- `abort()`
- `logout()`

Only after that should it expose higher-level browser, codegen, and BridgeRoot operations.

Migration Plan

1. Keep `gemstone-rs` independent and publishable.
2. Add a small PyO3 crate that depends on `gemstone-rs`.
3. Replace duplicated native loading code in `gemstone-py-native`.
4. Run the existing `gemstone-py` live tests through the Rust-backed native path.
5. Keep pure Python fallback behavior available.

The main design rule: Rust owns the native bridge; Python owns Python ergonomics.

Release Checklist

Use this checklist for coordinated crate, VSIX, and GitHub releases.

0.2.1 / Workbench 0.3.1 Notes

- CLI setup: global `--env-file`, `doctor --strict`, and safer `.env.gemstone-rs` template loading across CLI and explorer startup.
- Codegen: `codegen explain --json`, schema files, and a generated-wrapper compile smoke test.
- Mapping: missing keys, nested dictionaries, and array reads now report richer path context such as `booking.items[2]`.

- Explorer: setup assistant, env sample/write, and a structured Codegen Explain action.
- VS Code: `GemStone RS: Verify Strict Setup` and automatic env-file passing when `gemstoneRs.envFile` exists.

Workbench 0.3.2 Notes

- VS Code: `GemStone RS: Run Setup Assistant` renders the explorer `/api/setup/assistant` checks in the output panel.
- CI: schema copies and `codegen explain --json` output are validated by `scripts/validate_codegen_schemas.js`.
- Examples: the generated-wrapper smoke crate now runs `cargo test`.

Workbench 0.3.3 Notes

- CLI: `codegen explain-profile [--json] <profile-name> [profile-file]` resolves project profiles before rendering codegen summaries.
- VS Code: `GemStone RS: Codegen Explain` and `GemStone RS: Codegen Explain Profile` render structured classes, selectors, mappings, and test stubs.

0.2.2 / Workbench 0.3.4 Notes

- Core: project profile check reports now live in `gemstone-rs` and are shared by CLI, explorer, and VS Code.
- Explorer: `/api/codegen/profiles/check` returns the shared JSON report and the browser renders a profile status table with Preview, Diff, Check, and Generate actions.
- Tooling: `schemas/gemstone-rs.profile-check.schema.json` documents the shared report shape, and CI runs a real explorer HTTP endpoint smoke test.

Before Release

- Update crate versions in `Cargo.toml` files.
- Update `vscode-gemstone-rs-workbench/package.json`.
- Regenerate codegen examples:

```
cargo run -p gemstone-rs-cli -- codegen generate examples/codegen/gemstone-rs.codegen
cargo run -p gemstone-rs-cli -- codegen check-profile default examples/codegen/
gemstone-rs.codegen-profiles.json
cargo run -p gemstone-rs-cli -- profile validate examples/codegen/gemstone-
rs.codegen-profiles.json
cargo run -p gemstone-rs-cli -- profile validate --json examples/codegen/gemstone-
rs.codegen-profiles.json
cargo run -p gemstone-rs-cli -- profile list --json examples/codegen/gemstone-
rs.codegen-profiles.json
cargo run -p gemstone-rs-cli -- profile show default --json examples/codegen/
gemstone-rs.codegen-profiles.json
cargo run -p gemstone-rs-cli -- profile resolve default --json examples/codegen/
gemstone-rs.codegen-profiles.json
cargo run -p gemstone-rs-cli -- profile check examples/codegen/gemstone-rs.codegen-
profiles.json
cargo run -p gemstone-rs-cli -- profile check --json examples/codegen/gemstone-
rs.codegen-profiles.json
cargo run -p gemstone-rs-cli -- codegen explain-profile --json default examples/
codegen/gemstone-rs.codegen-profiles.json
cargo run -p gemstone-rs-cli -- profile sample > /tmp/gemstone-rs.codegen-
profiles.json
cargo test --manifest-path examples/codegen-wrapper-check/Cargo.toml
diff -u examples/codegen/gemstone-rs.codegen-profiles.json /tmp/gemstone-rs.codegen-
profiles.json
```

- Refresh screenshots when the explorer or workbench UI changed:

```
make screenshots
```

- Run local verification:

```
python3 scripts/version_check.py
python3 scripts/crate_metadata_check.py
make verify
make vscode-package
python3 docs/build_pdf_docs.py
```

Or use the dry-run release wrapper:

```
DRY_RUN=1 scripts/release_all.sh 0.2.2
```

`make verify` includes version and crate metadata checks, then checks that PDF generation completes and produces non-empty PDF files. The release wrapper writes repository-relative SHA256 entries and verifies the expected VSIX plus every PDF with `scripts/verify_release_artifacts.py`. `make verify` also runs an offline release-asset verifier smoke test so checksum mismatches and missing downloaded assets stay covered without contacting GitHub. The release workflow rebuilds and attaches fresh PDFs for the target runner because WeasyPrint output can differ byte-for-byte across platforms. The VSIX filename uses `vscode-gemstone-rs-workbench/package.json`; the crate release tag still uses the workflow `version` input.

Commit Review Pass

Before publishing, make a deliberate release commit review pass:

```
git status --short
git diff --stat
make verify
```

Confirm the release commit includes any regenerated PDFs, refreshed screenshots, profile schema changes, sample profile updates, and VSIX docs. Keep publishing for a separate step unless the release workflow is intentionally being run from that commit.

Publish

Set GitHub secrets once:

```
gh secret set CARGO_REGISTRY_TOKEN
gh secret set VSCE_PAT
```

Run the release workflow:

```
gh workflow run release.yml \
  --ref main \
  -f version=0.2.2 \
  -f publish-crates=true \
  -f publish-vsix=true \
  -f create-github-release=true \
  -f dry-run=false
```

The workflow publishes crates in dependency order:

1. `gemstone-gci`
2. `gemstone-rs-macros`

3. `gemstone-rs`
4. `gemstone-rs-cli`
5. `gemstone-rs-explorer`

It also packages/publishes the VSIX and can create the GitHub release with the VSIX attached.

Manual crate publishing remains available:

```
scripts/publish_crates.sh
```

For a dry run:

```
DRY_RUN=1 scripts/publish_crates.sh
```

Manual end-to-end publishing remains available through the release wrapper:

```
PUBLISH_CRATES=1 PUBLISH_VSIX=1 CREATE_GITHUB_RELEASE=1 VERIFY_PUBLIC=1 DRY_RUN=0
scripts/release_all.sh 0.2.2
```

The GitHub `Release` workflow now calls the same `scripts/release_all.sh` wrapper, so local and Actions releases share the same verification, packaging, checksum, publish, GitHub release, and post-publish verification path.

Dry-run mode checks each crate's publish file list locally. The real publish path still uses `cargo publish` in dependency order so crates.io resolves each new dependency after it has been published. If a workflow is rerun after a partial publish, already-published crate versions are skipped and the remaining crates continue.

Post-Release Verification

```
scripts/publish_verify.sh 0.2.2
```

The script checks:

- crates.io package/version JSON for all crates
- `cargo install gemstone-rs-cli`
- `cargo install gemstone-rs-explorer`
- `gemstone-rs --help`
- `gemstone-rs-explorer --help`
- VS Code Marketplace version matches `package.json`
- GitHub Release `v<version>` exists
- GitHub Release assets include the VSIX, `SHA256SUMS`, and every generated PDF
- downloaded GitHub Release assets match the published `SHA256SUMS`

To skip the GitHub Release asset check during early testing:

```
VERIFY_GITHUB_RELEASE=0 scripts/publish_verify.sh 0.2.2
```

Optional Live Smoke

Run the manual GitHub Actions live job with GemStone secrets configured, or run locally:

```
GS_RUN_LIVE_RUST=1 cargo test -p gemstone-rs live_ -- --test-threads=1
```

The live smoke coverage includes login/logout, `3 + 4 == 7`, global put/get, string round-trip, `perform`, commit, abort, browser lookup of `Object`, and generated wrapper `printString`.

The `--test-threads=1` flag is intentional. GemStone GCI has process-global session state, so live tests should not be run concurrently.

> This PDF was generated locally from `docs/`. If you edit the Markdown sources, rerun `python docs/build_pdf_docs.py`.